



The Ultimate Kubernetes Troubleshooting Guide

Find the Signals and Fix Kubernetes Failures Fast

You will learn to:

- understand why Pods won't start or keep crashing
- diagnose CPU, memory, and autoscaling failures
- debug networking, DNS, and traffic issues
- identify node-level failures and dangerous edge cases

Kubernetes rarely fails loudly.

Most of the time, it fails quietly:

- a Pod sits in **Pending** for no obvious reason
- a container starts, logs once, and disappears
- latency spikes, but nothing looks “down.”
- autoscaling behaves strangely, even though metrics seem fine

When this happens, Kubernetes already knows why.
The problem is knowing where to look.

This book helps you find that signal fast.

It's a practical troubleshooting guide for real production incidents, written for the moments when something is broken, and you need answers now.

In this book, you'll learn how to:

- understand why Pods won't start or keep crashing
- diagnose CPU, memory, and autoscaling failures
- debug networking, DNS, and traffic flow issues
- unblock storage, RBAC, and security context problems
- identify Node-level failures and rare but dangerous edge cases
- prevent many incidents before they reach production

If you've ever stared at **CrashLoopBackOff** at 3 AM, this book is for you.

Let's start by finding the signal.

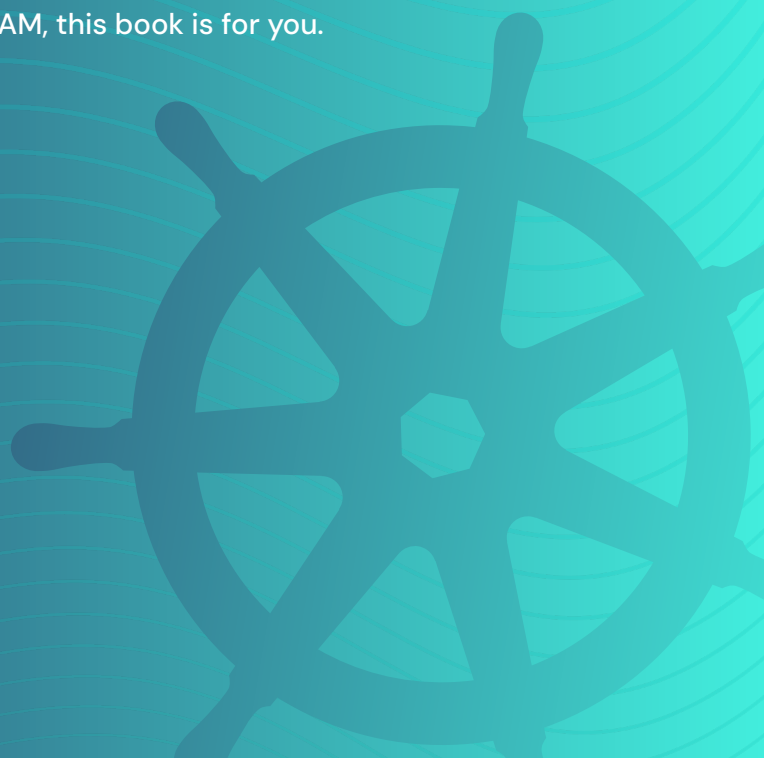


Table of Contents

Part 1: The Troubleshooting Toolkit	5
Chapter 1: The Troubleshooter's Toolbox.....	6
a. Kubernetes Events.....	6
b. Reading Logs with stern.....	7
c. Use k9s Dashboard	8
d. Built-in Tools.....	10
Part 2: The Pod Lifecycle (Startup and Runtime Problems).....	12
Chapter 2: Why Won't This Pod Start?	13
a. Image Pull Errors	13
b. Pods Stuck in Pending.....	15
c. Pods Stuck in ContainerCreating.....	16
Chapter 3: Why Pods Keep Crashing?	17
Part 3: Resources and Scaling (Performance and Stability Problems).....	19
Chapter 4: Requests vs Limits (The Most Common Mistake)	20
a. CPU Problems	20
b. Memory Problems.....	20
c. Quality of Service (QoS) Classes	21
Chapter 5: Quotas and Guardrails.....	22
a. ResourceQuota	22
b. LimitRanges	22
c. Noisy Neighbors on Shared Nodes.....	24
Chapter 6: Autoscaling Failures: When HPA Makes Things Worse.....	25
a. HPA Flapping	25
b. HPA and VPA.....	26
c. Missing Metrics (<unknown>).....	26
Part 4: Networking & Connectivity Issues	27
Chapter 7: Service Discovery & DNS.....	28
a. The ndots Problem	28
b. Debugging CoreDNS.....	28
c. Service Names vs Pod IPs	29
Chapter 8: Traffic Flow & Where Requests Disappear	30
a. Service Troubleshooting	30
b. Ingress Controllers & Gateway API.....	31
c. Network Policies	32

Part 5: Storage & Persistence	33
Chapter 9: The Persistent Volume Nightmare	34
a. PVCs Stuck in Pending	34
b. WaitForFirstConsumer Behavior	34
c. Attach and Detach Errors	35
d. Permission Denied on Mounted Volumes	36
Part 6: Security & Identity	37
Chapter 10: Access Denied (RBAC)	38
Chapter 11: Security Context Issues	39
a. "Operation Not Permitted" and Missing Capabilities	39
b. Read-Only Root Filesystems	40
c. Running as Non-Root and Port Binding	40
d. Pod Security Standards (Admission-Level Security)	41
Part 7: The Node Components	43
Chapter 12: The Kubelet	44
a. PLEG Issues	44
b. Eviction Manager	44
Chapter 13: Kube-Proxy & CNI	45
a. Kube-Proxy: IPTables vs IPVS	45
b. Debugging the CNI	45
c. IP Address Exhaustion	46
Part 8: Advanced and Rare Failures	47
Chapter 14: Zombies & Signals	48
a. Why PID 1 Matters?	48
b. Signals Propagation	48
c. Zombie Processes and Node Instability	49
d. Liveness Probes	49
Part 9: Real-World Case Studies	50
Case Study 1: Reddit's Pi-Day Outage	51
Case Study 2: Preply's Partial DNS Outage	51
Case Study 3: Free Now Node Bootstrap Failure	52
Part 10: Preventing These Problems at Scale	53
How PerfectScale Helps You Avoid Troubleshooting Pain?	54
1. Cluster-wide visibility (so you don't debug one Pod at a time)	54
2. Resource recommendations (to prevent the "requests vs limits" disasters)	56
3. Infrastructure Efficiency (to reduce node-level instability)	58
4. Alerts based on impact (so you catch trouble before users do)	60
5. Trends over time (to avoid slow-motion outages)	62
6. Safe automation (so right-sizing doesn't become a monthly project)	64

Part 1:

The Troubleshooting Toolkit

Before fixing any Kubernetes problem, you need to be in the right frame of mind and have the right starting point.

This section starts with the tools that work in every cluster and answers the most important question first: why did this happen?

The need for Troubleshooting Toolkit



why did this **happen?**



Chapter 1: The Troubleshooter's Toolbox

When something breaks in Kubernetes, this is the first place you have to look.

a. Kubernetes Events

When something fails in Kubernetes, an Event is usually created. Events are not logs from your application. They are messages from Kubernetes components like the scheduler, kubelet, and controllers. They explain the reasoning behind Kubernetes decisions, not just the current state.

If a Pod is stuck, failing, or restarting, Kubernetes has almost always recorded the reason as an Event.

The fastest way to understand is to go to your terminal and check:

```
kubectl get events -n production --sort-by='.metadata.creationTimestamp'
```

This command shows Events in time order across the namespace. It lets you see what happened just before things broke and what Kubernetes tried next.

Some Events appear again and again. Once you recognize them, troubleshooting becomes much faster:

FailedScheduling means the scheduler could not find a suitable node. This is almost always about resource requests, node selectors, taints, or affinity rules. The Pod never even reached a node. We'll dig deeper into scheduling problems in Part 2.

FailedMount means the Pod was placed on a node, but Kubernetes could not attach or mount the requested persistent storage. This refers to block storage (like cloud disks such as AWS EBS, GCE PD, Azure Disk) or on-prem storage provisioned via a StorageClass. Until this is fixed, the container will never start. Storage issues get their own section later in Part 5, but Events will always point you there first.

BackOff means Kubernetes is slowing down restarts because the container keeps failing. This points to application startup problems. You'll see this one a lot in Chapter 3 when we talk about crashes.

NodeNotReady means the node itself is unhealthy. Pods on that node may be evicted or stuck. This is a node problem, not an app problem. Node-level issues are covered in Part 7.

You can watch Events as they happen:

```
kubectl get events --watch
```

This is useful when deploying new workloads or debugging repeated failures. You can see scheduling decisions, retries, and errors in real time.

b. Reading Logs with stern

Events tell you what Kubernetes decided. Now you need to know what your application was doing when things went wrong.

Logs are the next thing people check after Events. The problem is that **kubectl logs** does not work well once Pods start restarting or scaling. You need the exact Pod name. When a Pod crashes, a new one appears with a new name, and the logs you were following are gone. In a CrashLoopBackOff, this becomes frustrating very quickly.

This is where stern helps. Stern is a free, open-source command-line utility that runs on your local terminal and connects to your cluster using your existing kubeconfig. It does not require any cloud account, paid license, or external service, and logs/data never leave your workstation or cluster.

Stern simply watches the cluster and streams logs from Pods based on a name pattern, not a single Pod. You stop worrying about Pod names and start focusing on the workload.

With stern, you can stream logs from all matching Pods at once. For example:

```
stern backend
```

This command shows logs from every Pod whose name contains **backend**. When a Pod crashes and a new one starts, stern automatically attaches to the new Pod. You don't miss startup logs.

This makes the stern useful for debugging CrashLoopBackOff issues, Startup failures, and short-lived containers.

Another big advantage is catching logs right before a crash. Many apps fail fast: they log one error and exit. **kubectl logs** misses this window. Stern stays attached and prints logs the moment the container starts, which is often where the real error appears.

In many deployments, Pods run more than one container. An app container may run alongside a sidecar, such as a service mesh proxy. Stern can stream logs from all containers in the Pod, so you can see how the app and the sidecar behave simultaneously. This shared view explains issues that are invisible when logs are viewed in isolation.

If Events tell you *why Kubernetes is unhappy*, stern shows you what the application was doing when it failed.

On macOS, stern can be installed using Homebrew:

```
brew install stern
```

On Linux, you can download the binary and place it in your PATH.

Once installed, you can verify it by running:

```
stern --version
```

If this works, stern is ready to use with any Kubernetes cluster your kubeconfig points to.

For installation options on other platforms and advanced usage, refer to the official stern repository.

c. Use k9s Dashboard

Now that you know how to read Events and follow logs, let's talk about speed.

When you are troubleshooting a live issue, speed matters. Typing long **kubectl** commands slows you down, especially when Pods are restarting or nodes are under pressure. k9s gives you a real-time view of the cluster, so you can move fast without losing context.

k9s is an open-source, terminal-based UI for Kubernetes; it runs entirely in your terminal window and is not a browser-based dashboard. k9s uses the Kubernetes API, just like **kubectl**. It does not change cluster behavior. It simply helps you see and navigate what is already there.

Before using k9s, you need to install it on your local machine. Like stern, k9s is a client tool that lives on your local machine, uses your kubeconfig, and never sends data to a third-party server.

If kubectl already works for a cluster, k9s will work for that cluster too.

On macOS, k9s can be installed using Homebrew:

```
brew install k9s
```

On Linux, you can download the binary and place it in your PATH.

After installation, you can verify it by running:

```
k9s version
```

Once installed, you can start k9s simply by running:

```
k9s
```

This opens the dashboard for the current Kubernetes context.

For other platforms and installation methods, refer to the official k9s documentation.

Inside k9s, common actions are one key away. You can jump between resources without retyping names or flags.

Press **l** to view logs, **d** to describe a resource, and **s** to open a shell in a container. This makes it easy to move from issues to details in seconds.

When a Pod is failing, this flow is much faster than running separate commands. You'll use `kubectl describe` and `logs` constantly throughout this book – k9s just makes those same workflows faster.

Also, k9s' Pulse mode is useful when you suspect resource pressure. It shows live CPU and memory usage for nodes and Pods. You can quickly spot spikes and see whether a problem is local to one Pod or affecting the whole node. This is enough to confirm a resource issue before digging deeper.

k9s makes it easy to delete Pods on purpose. Restarting a Pod is a common way to test recovery behavior. With k9s, you can do this quickly without copying long Pod names.

If the Pod comes back healthy, you know the system can recover. If it doesn't, you know where to focus next.

Note: k9s itself does not change how Kubernetes behaves, it can perform actions through the Kubernetes API just like running `kubectl delete`. Any deletion or modification you trigger in k9s is an explicit action you choose, not something automatic.



Source: <https://k9scli.io/assets/screens/pulses.png>

d. Built-in Tools

In many production clusters, you are not allowed to install extra tools. When access is limited, these tools become your safety net.

The first command you should reach for is **kubectl describe**.

It gives you the current state of a Pod or Node, along with recent Events. This is faster than reading logs.

For a Pod, **kubectl describe pod** shows why the Pod is Pending or not Ready, container states and restart reasons, failed mounts, image pull errors, and probe failures.

For a Node, **kubectl describe node** tells you if the node is under pressure, tainted, or NotReady. Many issues can be identified here. In parallel, **kubectl get events** helps you understand what Kubernetes tried to do and why it failed, as we discussed this already.

Sometimes logs are not enough, especially when containers are distroless and don't have a shell. This is where **kubectl debug** helps. Kubernetes allows you to attach an ephemeral container to a running Pod for debugging.

An ephemeral container runs alongside the existing containers, shares the same network and volume, and does not change the Pod spec.

This makes it possible to inspect files, check network connectivity, or run debugging tools without rebuilding the image.

There are a few cases where the problem is not inside the Pod at all, but on the Node itself. Examples include disk pressure, network failures, or broken CNI plugins. In such cases, you need to debug the host.

You can use tools like **nsenter**, you to enter the node's namespaces and inspect host-level processes, mounts, and networking. This is usually done through a privileged container or direct node access.

However, **nsenter** requires root-level access to the node. In most production environments – especially managed clusters like EKS, GKE, or AKS – direct SSH or root access to nodes is restricted for security reasons. This means **nsenter** is often not practical in real-world cloud environments.

In these cases, the recommended approach is to use **kubectl debug node/<node-name>**, which launches a temporary debugging Pod on the node. This method uses Kubernetes' built-in debugging capabilities instead of relying on direct host access.

If SSH access is allowed, always follow your cloud provider's recommended secure access method (for example, Identity-Aware Proxy in GCP or Instance Connect in AWS) instead of enabling open SSH access.

When debugging nodes, always respect security boundaries. Production clusters are designed to limit direct node access on purpose.

```
[rono@fedora:~]$ kubectl-nsexter -u vagrant httpbin-5876b4fbc9-rtvrq ip -c a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
3: eth0@if10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether de:11:26:18:ce:f4 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.100.6.67/32 brd 10.100.6.67 scope global eth0
        valid_lft forever preferred_lft forever
[rono@fedora:~]$ kubectl-nsexter -u vagrant httpbin-5876b4fbc9-rtvrq ss -tln
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
LISTEN     0            128         0.0.0.0:80               0.0.0.0:*
```

Source: <https://github.com/pabateman/kubectl-nsexter>

Part 2:

The Pod Lifecycle

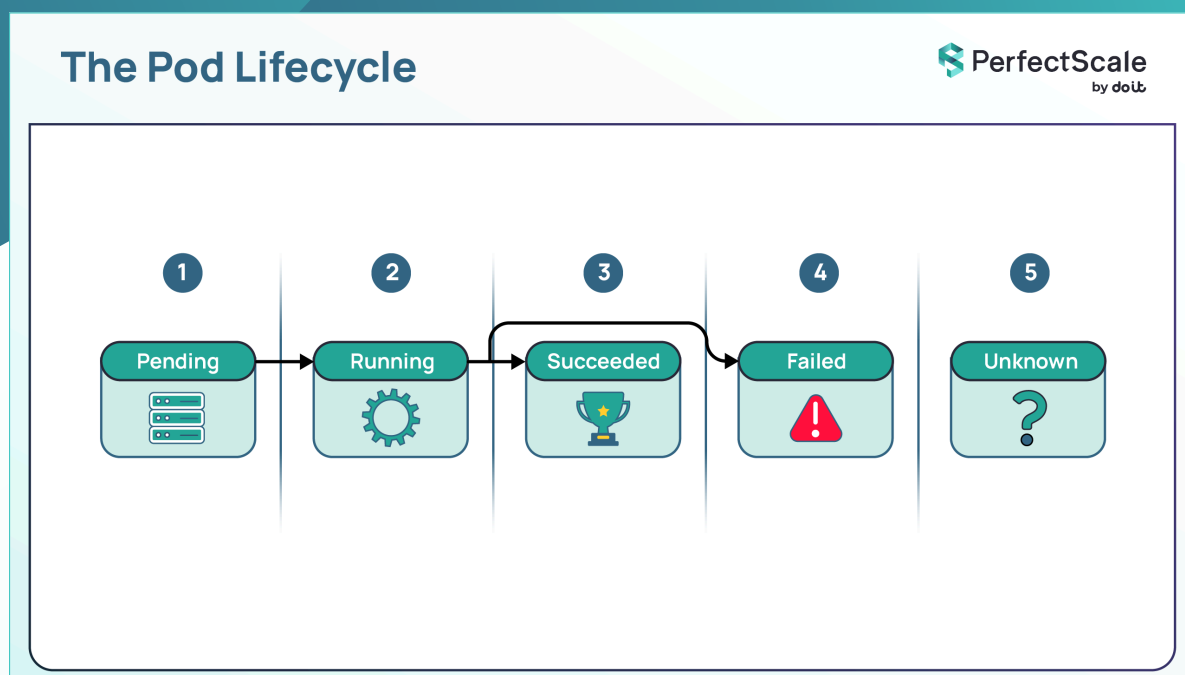
Startup and Runtime Problems

Now that you know how to investigate, let's talk about what you're actually investigating.

Most Kubernetes issues happen very early in the Pod lifecycle. Either the Pod never starts, or it starts and then keeps crashing.

This section focuses on those Day-1 problems that block applications from running at all.

If a Pod is not **running** and **Ready**, Kubernetes is already telling you why. You just need to know where to look and how to read the signals.



Chapter 2: Why Won't This Pod Start?

When a Pod does not start, it usually fails in one of three places: pulling the image, getting scheduled, or creating the container. Let's discuss:

a. Image Pull Errors

These errors (ImagePullBackOff, ErrImagePull) mean Kubernetes could not download the container image. The Pod never reaches the scheduling or startup phase.

```
test-pod    0/1    ErrImagePull    0    19s
test-pod    0/1    ImagePullBackOff 0    52s
```

The most common cause is a wrong image name or tag. A small typo is enough to break the deployment.

Another common reason is missing registry credentials, especially for private images. Kubernetes will keep retrying and eventually show ImagePullBackOff.

Public registries can also block you. Docker Hub applies pull rate limits, and when those limits are hit, image pulls start failing even though nothing changed in your configuration.

User type	Pull rate limit per 6 hours	Number of public repositories	Number of private repositories
Business (authenticated)	Unlimited	Unlimited	Unlimited
Team (authenticated)	Unlimited	Unlimited	Unlimited
Pro (authenticated)	Unlimited	Unlimited	Unlimited
Personal (authenticated)	200	Unlimited	Up to 1
Unauthenticated users	100 per IPv4 address or IPv6 /64 subnet	Not applicable	Not applicable

Let's discuss how to troubleshoot ImagePull Errors:

The first thing to do is describe the Pod:

```
kubectl describe pod test-pod
```

This shows the exact reason why the image pull failed.

Case 1: Wrong image name or tag

You may see Events like:

```
Failed to pull image "myapp:v2": manifest for myapp:v2 not found
```

This means the registry was reachable, but the image or tag does not exist.

In this case, check the image name and tag in the Pod or Deployment, confirm the image exists in the registry fix the typo and redeploy.

Case 2: Registry unavailable or rate-limited

You may see Events like:

```
Failed to pull image "nginx:latest": toomanyrequests  
You have reached your pull rate limit
```

or network-related errors like:

```
Failed to pull image: dial tcp: lookup registry-1.docker.io: no such host
```

This means Kubernetes could not reach the registry.

In this case, you can wait for rate limits to reset, use a private registry or mirror, check node-level network connectivity or any firewall blocking the node outbound connections.

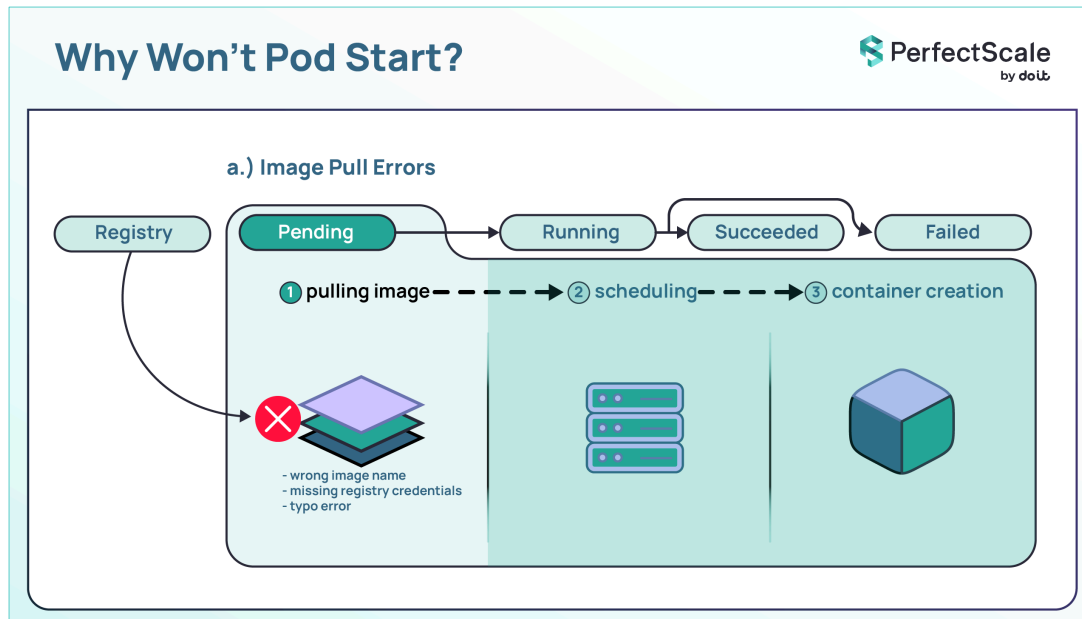
Case 3: Missing or incorrect registry credentials

You may see Events like:

```
Failed to pull image "private-registry/app:1.0": unauthorized  
authentication required
```

This means Kubernetes reached the registry but was denied access.

In this case, you can create or fix the imagePullSecret, ensure the secret exists in the same namespace, and then reference it in the Pod or ServiceAccount.



b. Pods Stuck in Pending

If the image pulls successfully, the next step is scheduling.

A Pod in Pending state means the scheduler cannot find a Node that satisfies all requirements.

Sometimes the cluster simply does not have enough CPU or memory. In other cases, the Pod requests are too large compared to what the nodes can provide. The scheduler treats requests as hard requirements. We'll cover resource requests and limits in much more detail in Part 3 – for now, just know they're a common reason Pods stay Pending.

Scheduling can also be blocked by taints, tolerations, and node affinity. A Pod may technically fit on a node, but rules prevent it from being placed there.

Example: taints and tolerations

If a node has a taint like:

```
node-role.kubernetes.io/control-plane:NoSchedule
```

If the Pod does not have a matching toleration, the scheduler will refuse to place it.

Then, you may see:

```
0/3 nodes are available: node(s) had taint
{node-role.kubernetes.io/control-plane: NoSchedule}
```

Example: node affinity

If a Pod requires a specific node label that does not exist, scheduling will fail.

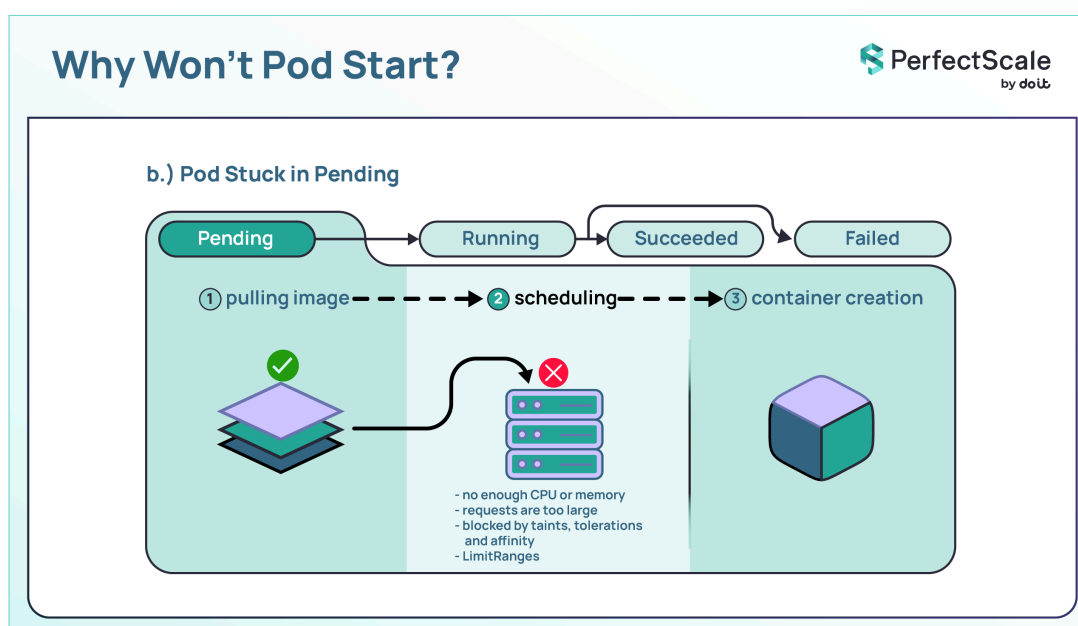
You may see Events like:

```
0/5 nodes are available: node(s) didn't match node affinity
```

In this case, check Pod nodeSelector and affinity rules, check node labels, and verify required tolerations exist.

Another common and missed issue is LimitRanges. They can silently add default resource values. If those defaults exceed available capacity, the Pod will stay Pending even though you never set those values yourself.

```
test-pod 0/1 Pending 0 52s0
```



c. Pods Stuck in ContainerCreating

If a Pod is scheduled but never reaches Running, it is usually stuck in ContainerCreating.

```
NAME                READY  STATUS             RESTARTS  AGE
test-pod-c85f7f79c-99khh  0/1   ContainerCreating  0         35m
```

One common reason is a CNI problem. If the cluster runs out of IP addresses or the CNI plugin fails, the container cannot get networking and will never start. (Networking gets its own deep dive in [Part 4](#).)

```
Events:
Failed to create pod sandbox: failed to assign IP address
```

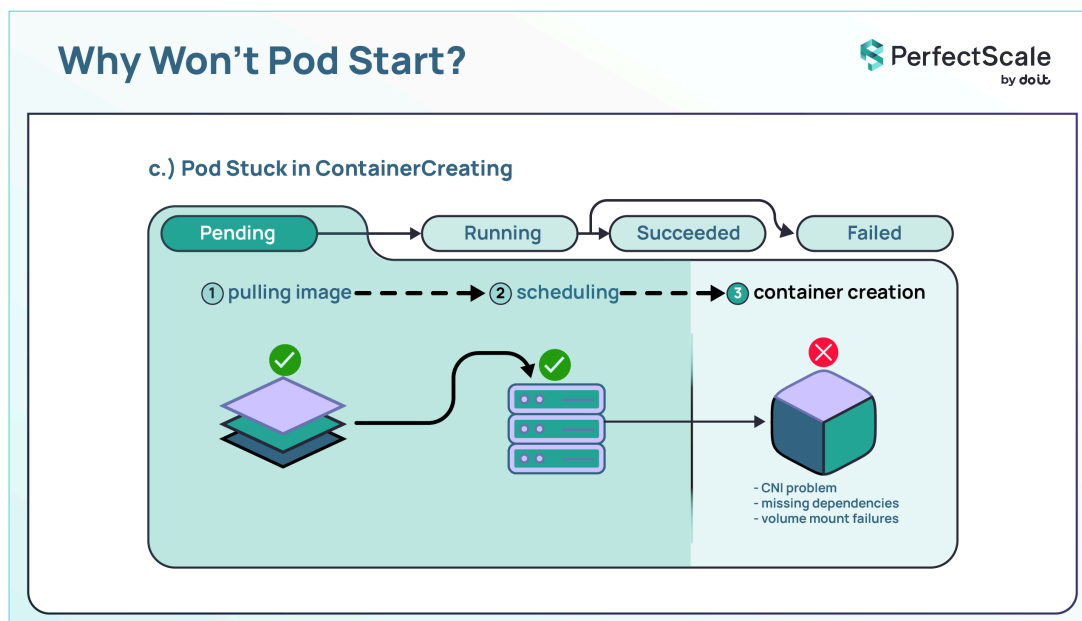
Another issue is missing dependencies. If a Pod references a ConfigMap or Secret that does not exist, Kubernetes will keep waiting.

```
Events:
  ConfigMap "app-config" not found
```

The same happens with volume mount failures, especially when storage is misconfigured or unavailable.

```
Events:
  Unable to attach or mount volumes: timed out waiting for the condition
```

In all these cases, Events clearly explain what is blocking container creation.



Chapter 3: Why Pods Keep Crashing?

If a Pod starts but does not stay alive, Kubernetes marks it as **CrashLoopBackOff**. This means the container exists repeatedly, and Kubernetes backs off before restarting it again.

```
busybox    0/1    CrashLoopBackOff    18 (2m12s ago)    70m
```

When you see *CrashLoopBackOff*, the most important step is to inspect the container state and exit codes.

```
kubectl describe pod busybox
```

To understand why this happens, you need to look at container exit codes.

Code 0: means the application exited successfully, but Kubernetes expected it to keep running. This points to a wrong command or restart policy.

Codes 1 or 255 usually indicate an application error. The logs are the only reliable source to understand what failed.

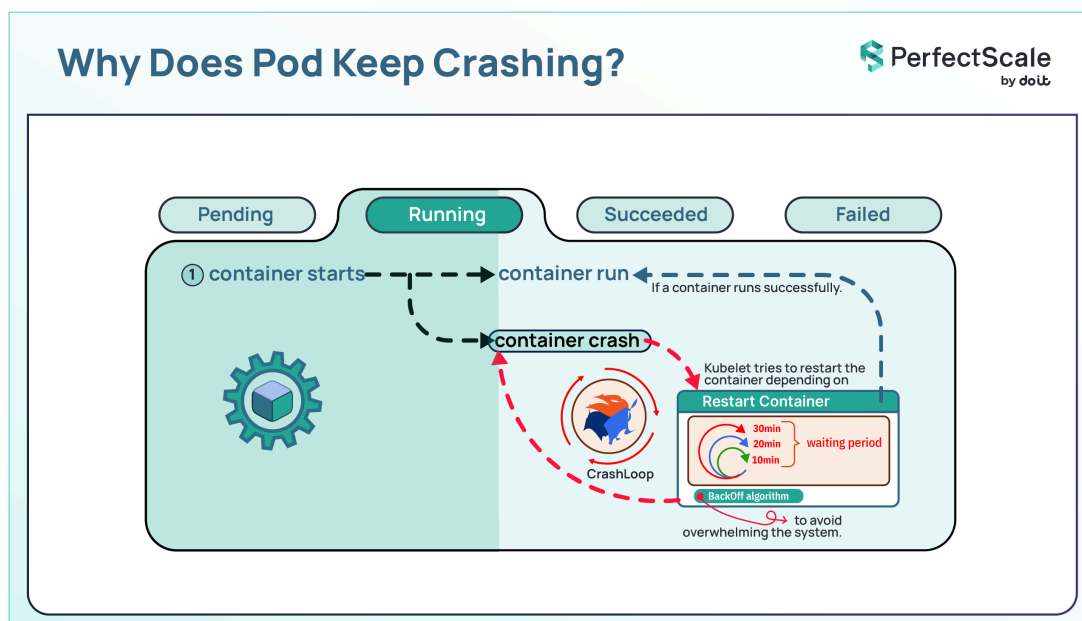
Code 137: means the container was killed due to memory pressure. This happens when the container exceeds its memory limit, and the Linux kernel terminates it.

Code 143: means the container received a SIGTERM and shut down gracefully. This happens during rolling updates or when Pods are being evicted.

```
State:      Terminated
Reason:     Error
Exit Code:  1
Started:    Sun, 11 Jan 2026 15:12:47 -0800
Finished:   Sun, 11 Jan 2026 15:12:49 -0800
Last State: Terminated
```

Also, one important thing to note is that InitContainers run before the main container starts. If an InitContainer fails, the main container never runs. Kubernetes will keep retrying the InitContainer, and the Pod will appear stuck.

This can create confusion. The Pod looks alive, but nothing progresses. Always check the InitContainer status when a Pod never reaches **Running**.





Part 3:

Resources and Scaling

Performance and Stability Problems

Many Kubernetes issues are not bugs in the application. They are resource problems. In Part 2, we focused on why Pods fail to start or keep crashing. In this part, the Pods are already running, but are slow, unstable, or suddenly killed.

This means the Pod is not getting enough CPU or memory, or it is hitting the limits that were set for it.

Let's understand how Kubernetes handles resources.

Chapter 4:

Requests vs Limits (The Most Common Mistake)

The most important rule in Kubernetes resource management is simple:

Requests decide where a Pod is scheduled.

Limits decide how much is allowed to be used at runtime.

These two values control very different parts of the system, but they are misunderstood.

a. CPU Problems

CPU is a compressible resource. When a container reaches its CPU limit, it is throttled, not killed. This is why applications under CPU pressure feel slow instead of crashing.

Kubernetes uses Linux CFS quotas to enforce CPU limits. When a container uses its allowed CPU time for a given period, it is paused until the next period starts. This pause shows up as latency in the application.

A clear signal of this problem is high CPU throttling time. If throttled time keeps increasing while CPU usage looks normal, the container is being slowed down by its limit.

In some performance-sensitive workloads, CPU throttling is unacceptable. Kubernetes provides a *static CPU manager policy* for these cases. It pins containers to specific CPU cores and avoids throttling. This should only be used for workloads that truly need it, because it reduces scheduling flexibility and cluster efficiency.

```
cpuManagerPolicy: "static"
```

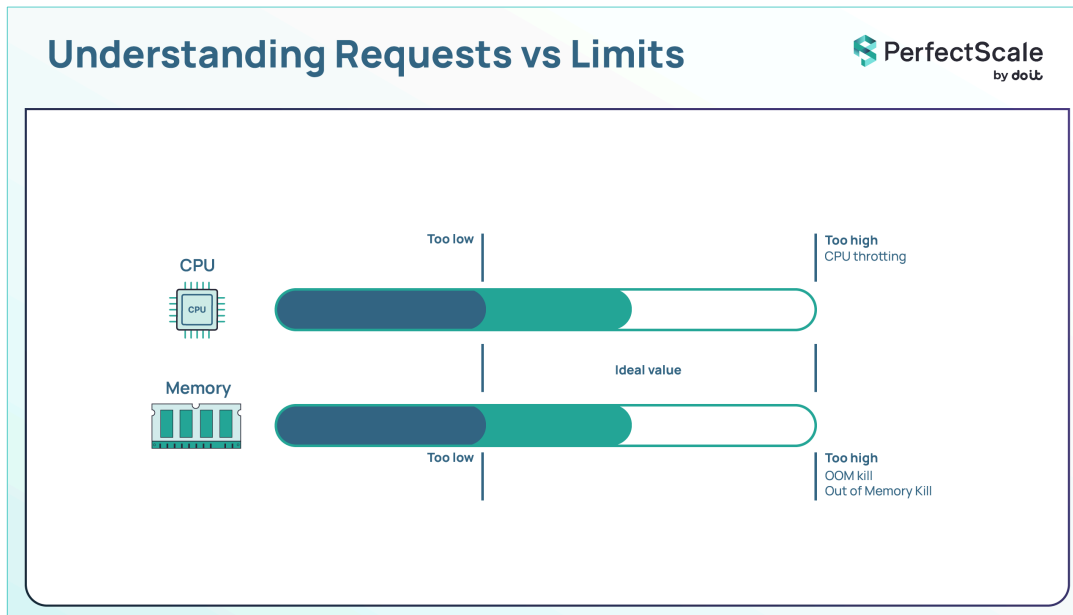
b. Memory Problems

Memory behaves very differently from the CPU. Memory limits are hard limits.

When a container uses more memory than its limit, the Linux kernel immediately kills the process. Kubernetes reports this as **OOMKilled**. There is no throttling and no warning.

This is why memory-related failures are sudden and disruptive.

Java applications hit this problem frequently. The JVM may think it has access to more memory than the container actually allows. If the JVM heap is larger than the container memory limit, the process will be killed even though the application itself looks healthy. Container-aware JVM settings are critical to avoid this, and PerfectScale continuously analyzes the behavior and JVM patterns of your applications and autonomously optimizes resources.



c. Quality of Service (QoS) Classes

Kubernetes assigns every Pod a QoS class based on its resource requests and limits.

Pods with requests and limits for both CPU and memory are guaranteed. These Pods get the strongest protection.

Pods with requests but with no limits are Burstable. Most workloads fall into this category.

Pods without requests or limits are BestEffort. These Pods are the first to suffer when the node is under pressure.

When a Node runs out of resources, Kubernetes must evict Pods. BestEffort Pods go first, then Burstable Pods. Guaranteed Pods are evicted last.

Understanding Quality of Service(QoS) Classes

PerfectScale by doit

QoS Class	Resource Requests	Resource Limits	Priority
Guaranteed	requests == limits (for all containers)	Required	Low
Burstable	At least one container with requests < limits	Optional	Medium
BestEffort	No requests or limits	None	High

Chapter 5: Quotas and Guardrails

Sometimes Kubernetes blocks a Pod before it even starts. This happens because of quotas and guardrails that are applied at the namespace or node level.

a. ResourceQuota

A ResourceQuota limits how much a namespace can use. This includes CPU, memory, number of Pods, and even objects like Services or PVCs.

When a namespace hits its quota, new Pods simply fail to be created. The scheduler is never involved. You might see an error during deployment, or the Pod may not appear at all.

The important thing to understand is that this is not a scheduling problem. Even if the cluster has plenty of free resources, Kubernetes will reject the Pod because the namespace is not allowed to use more.

Whenever Pods fail to create without a clear reason, always check if a ResourceQuota is in place.

Run:

```
kubectl describe quota -n example-namespace
```

You may see Events like:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Warning	ExceededQuota	12s	kube-apiserver	exceeded quota: high-priority-quota

This confirms that Kubernetes rejected the Pod before scheduling.

In this case, you should increase the quota if the namespace legitimately needs more resources, delete unused Pods or objects, or move the workload to a different namespace.

b. LimitRanges

LimitRanges can automatically add default CPU and memory requests or limits to every Pod in a namespace.

This becomes a problem when the defaults are too high. A Pod that looks small but ends up with large requests after LimitRanges are applied. The scheduler then cannot place it anywhere, and the Pod stays Pending.

Because these values are added automatically, many engineers waste time debugging the wrong thing. Always check the final resource values on the Pod, not just what you wrote in YAML.

Run:

```
kubectl describe pod <pod-name>
```

You may see unexpected values:

```
Requests:
cpu:    1000m
memory: 2Gi
Limits:
cpu:    2000m
memory: 4Gi
```

Check Events:

```
Events:
Type      Reason      Age   From          Message
----      -
Warning   FailedScheduling 1m    default-scheduler 0/5 nodes available:
insufficient CPU
```

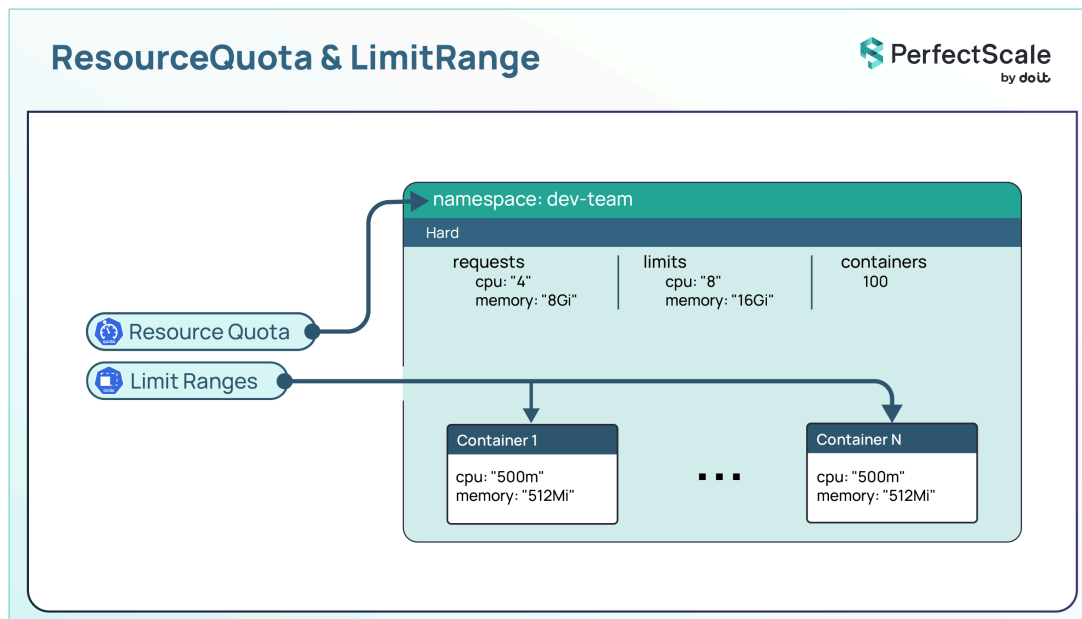
Also check:

```
kubectl describe limitrange -n <namespace>
```

You may find:

```
Default Request:
cpu:    1000m
memory: 2Gi
```

In this case, lower the default values in the LimitRange or explicitly set smaller requests and limits in your Pod spec.



c. Noisy Neighbors on Shared Nodes

In shared clusters, multiple Pods run on the same Node. When resource limits are not set correctly, one Pod can consume more than its fair share and hurt others. This is known as the noisy neighbor problem.

CPU overuse leads to throttling and latency. Memory overuse leads to evictions and OOMKills. Pods with lower QoS are hit first, even if they are not the cause of the problem.

To detect this, look for nodes under pressure and Pods with high usage compared to their requests. Without guardrails, one misbehaving workload can destabilize an entire node.

Run:

```
kubectl describe node <node-name>
```

You may see:

```
Conditions:
  Type          Status Reason
  ----          -
  MemoryPressure True  KubeletHasInsufficientMemory
```

And Events like:

```
Events:
  Type    Reason    Age   From    Message
  ----    -
  Warning Eviction  30s   kubelet Evicting pod due to memory pressure
```

Then inspect affected Pods:

```
kubectl describe pod <pod-name>
```

```
Last State:
  Terminated:
    Reason:  OOMKilled
    Exit Code: 137
```

To solve this, set proper CPU and memory requests and limits, use QoS classes intentionally, and avoid running critical and best-effort workloads together.

Chapter 6: Autoscaling Failures: When HPA Makes Things Worse

This chapter focuses on the most common autoscaling failures seen in production.

a. HPA Flapping

HPA flapping happens when replicas keep increasing and decreasing in short cycles. One minute the app scales up, the next minute it scales back down. This creates churn, increases latency, and can even trigger outages.

The most common cause is unstable metrics. CPU spikes for a short time, HPA reacts, then usage drops again. Another common reason is aggressive scaling settings, such as very low stabilization windows or poorly chosen target values.

HPA reacts only to what it sees. If the metric is bad, scaling will be bad/noisy too. Always remember: autoscaling makes bad signals worse.

Run:

```
kubectl describe hpa <hpa-name>
```

You may see repeated Events:

```
Events:
  Type     Reason           Age   From                Message
  ----     -
  Normal   SuccessfulRescale 1m    horizontal-pod-autoscaler New size: 10
  Normal   SuccessfulRescale 40s   horizontal-pod-autoscaler New size: 4
```

In this case, increase stabilization windows (`stabilizationWindowSeconds`), use more stable metrics, and try to avoid very low target utilization values.

b. HPA and VPA

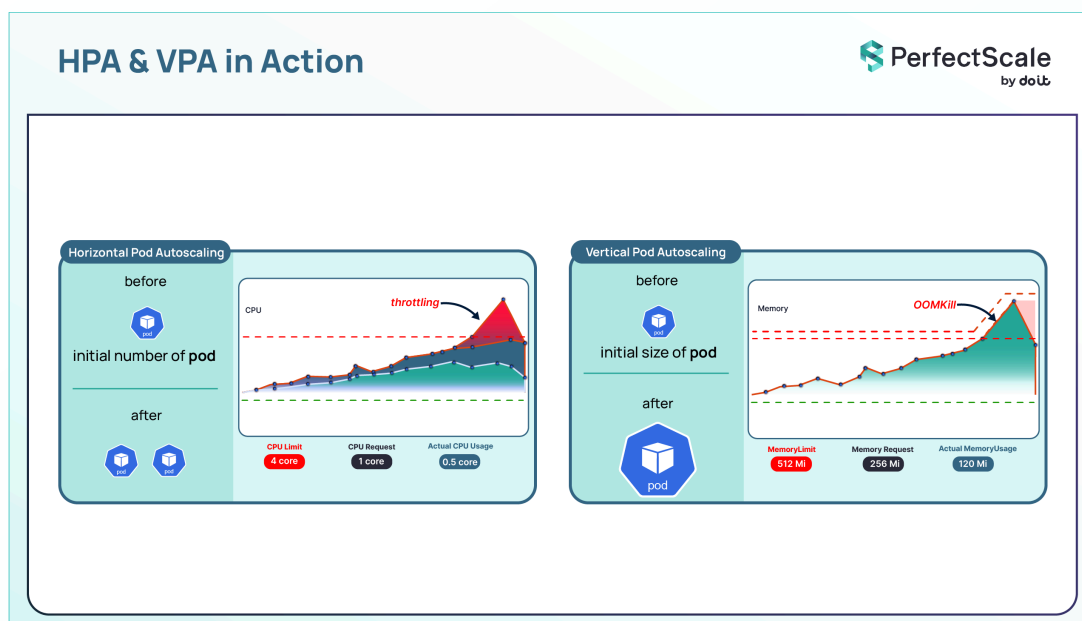
HPA and VPA solve different problems.

HPA changes the number of Pods based on metrics like CPU or memory usage.

VPA changes the resource requests and limits of Pods.

When both operate on the CPU or memory at the same time, a feedback loop forms. VPA changes requests, HPA recalculates utilization, and scaling decisions become unstable. This is why running HPA and VPA together on CPU or memory is not recommended.

If you need both, the good pattern is: HPA for replica count and VPA in recommendation-only mode, or for resources not used by HPA. Also, another option is using smart HPA-aware vertical autoscaling as provided by PerfectScale.



c. Missing Metrics (<unknown>)

When HPA shows **<unknown>** instead of numbers, it means metrics are not available. Without metrics, HPA cannot make decisions, and scaling stops.

This usually points to issues with the Metrics Server. The common causes can be Metrics Server not installed or not reachable, Kubelet authentication or TLS issues, and Pods starting before metrics are ready.

Metrics Server provides resource metrics only. If it fails, CPU- and memory-based autoscaling will not work, even though the cluster itself looks healthy.

Always confirm that metrics are being collected before debugging HPA logic.



Part 4:

Networking & Connectivity Issues

Up to this point, we focused on Pods, resources, and scaling. But Networking issues are the hardest to troubleshoot in Kubernetes.

This section focuses on service discovery, name resolution, and traffic flows.

Chapter 7: Service Discovery & DNS

Kubernetes relies on DNS for almost all internal communication. Pods talk to Services by name, not by IP. If DNS is slow or broken, applications look healthy but fail to connect to each other.

Before blaming the network, always confirm that DNS works as expected.

a. The **ndots** Problem

By default, Kubernetes sets **ndots:5** in Pod DNS configuration. This means that any name with fewer than five dots is treated as a potentially internal name.

When an application tries to resolve an external domain like **api.example.com**, Kubernetes may first try multiple internal DNS paths before going to the real external DNS server. Each failed attempt adds latency.

This is a common cause of slow outbound requests from Pods. The application is not slow—the DNS lookup is.

Therefore, understanding **ndots** is important when debugging unexplained delays in external calls.

```
cat /etc/resolv.conf
nameserver 10.10.10.10
search <namespace>.svc.cluster.local svc. cluster.local cluster.local
options ndots:5
```

b. Debugging CoreDNS

CoreDNS is the DNS server for the cluster. If name resolution fails or behaves strangely, CoreDNS is the first place to look.

The common signs of CoreDNS issues include: intermittent resolution failures, timeouts when resolving service names, and high DNS latency under load.

CoreDNS logs and metrics reveal the problem quickly. Misconfigured upstream DNS servers or overloaded CoreDNS Pods are frequent causes.

To inspect CoreDNS logs, run:

```
kubectl logs -n kube-system -l k8s-app=kube-dns
```

You might see log entries like:

```
[ERROR] plugin/errors: 2 kubernetes.default.svc.cluster.local. A: read udp 10.244.0.10:5353->8.8.8.8:53: i/o timeout
```

This usually indicates that CoreDNS cannot reach its upstream DNS server.

Another example:

```
[ERROR] plugin/errors: 2 myservice.default.svc.cluster.local. AAAA: no such host
```

This can indicate that the Service does not exist or the record was not created properly.

If logs show repeated timeouts under load, CoreDNS itself may be overloaded.

c. Service Names vs Pod IPs

In Kubernetes, Service names are stable, but Pod IPs are not. Pods are replaced, rescheduled, and restarted all the time. If an application connects directly to a Pod IP, that connection will eventually break.

Services exist to provide a stable endpoint. DNS resolves the Service name to a virtual IP, and Kubernetes handles routing to the correct Pod.

If connectivity works with Pod IPs but fails with Service names, the issue is almost always DNS or Service configuration. If it works with Service names but not Pod IPs, the application may be bypassing Kubernetes networking.

When troubleshooting DNS from the workload side, follow these steps:

1. Verify the Service exists:

```
kubectl get svc <service-name>
```

2. Verify the Service has Endpoints:

```
kubectl get endpoints <service-name>
```

3. Test DNS resolution from inside a Pod:

```
kubectl exec -it <pod-name> -- nslookup <service-name>
```

If DNS resolution works but requests still fail, the issue is likely routing or application-level, not DNS.

Under heavy load, DNS can become a bottleneck. Kubernetes supports horizontal autoscaling of CoreDNS via a deployment object.

If DNS latency increases during traffic spikes, verify that CoreDNS replicas scale appropriately:

```
kubectl get deployment coredns -n kube-system
```

If replica count stays low while cluster size or traffic grows, DNS queries may begin timing out.

So, scaling CoreDNS horizontally helps maintain reliable service discovery as the cluster grows.

Chapter 8: Traffic Flow & Where Requests Disappear

Pods are running. Services exist. Yet requests still time out or return errors. This happens because traffic flows through several layers, and a small mistake in any layer can break the whole path.

This chapter focuses on where traffic commonly gets lost.

a. Service Troubleshooting

A Service only routes traffic to Pods that match its selector. If the selector does not match any Pod labels, the Service has no Endpoints. This is the most common Service issue in Kubernetes.

The Service still exists, DNS still resolves, but traffic goes nowhere. Requests hang or fail silently. Always check that the Service selector exactly matches the Pod labels.

Run:

```
kubectl describe svc <service-name>
```

If you see:

```
Endpoints: <none>
```

This means the Service selector does not match any Pod labels.

Check Pod labels:

```
kubectl get pods --show-labels
```

If labels don't match the Service selector, traffic will never reach the Pods.

Another common mistake is confusing **port** and **targetPort**.

The **port** is the port exposed by the Service.

The **targetPort** is the port the container actually listens on.

If these do not match, traffic reaches the Pod but never reaches the application. The Pod looks healthy, but connections fail. This problem is especially common when container ports change, and the Service is not updated.

b. Ingress Controllers & Gateway API

Ingress sits in front of Services and adds another failure point. Different errors usually point to different problems.

A *502 Bad Gateway* usually means the Ingress controller cannot reach the backend Pod. This happens when the Service has no Endpoints or the target port is wrong.

```
Endpoints: <none>
```

In this case, fix service selectors, fix service targetPort, and make sure Pods are running and ready.

A *503 Service Unavailable* usually means the Service exists, but there are no healthy Pods behind it. Readiness probes are often the cause here.

```
Warning: Unhealthy kubelet Readiness probe failed
```

To solve this, fix the readiness probe path, port, or timeout, and ensure the application is actually ready before receiving traffic.

TLS issues are another source of trouble. If certificates are missing, expired, or incorrectly configured, connections fail during the handshake. It looks like random connection problems or browser errors from the outside, but the app itself is fine.

While Ingress is still widely used, Kubernetes now promotes the Gateway API as the next-generation standard for managing traffic.

Gateway API introduces resources such as **Gateway**, **HTTPRoute**, and **TCPRoute**. Instead of defining routing rules directly inside an Ingress object, traffic policies are split into clearer, more flexible components.

This makes routing more expressive, but it also introduces new places where misconfiguration can break traffic.

If traffic is failing and you are using Gateway API, check:

```
kubectl get gateways
kubectl describe gateway <gateway-name>
```

Look for conditions such as **Accepted: False** or **Programmed: False**. These conditions indicate that the Gateway controller could not apply the configuration.

The common issues can be a route not attached to a valid Gateway, hostname mismatch, a backend reference pointing to a non-existing service or backend service with no endpoints.

c. Network Policies

NetworkPolicies control which Pods can talk to each other. When they are misconfigured, traffic is dropped silently. There is no error message, no log, just timeouts.

If traffic works within a namespace but fails across namespaces, a NetworkPolicy is often the reason. Always confirm that both ingress and egress rules allow the traffic you expect.



Part 5:

Storage & Persistence

This chapter focuses on the most common persistent volume problems and how to recognize them quickly.

Chapter 9: The Persistent Volume Nightmare

a. PVCs Stuck in Pending

When a PersistentVolumeClaim stays in **Pending**, it means Kubernetes cannot find a matching volume. In most cases, this comes down to a StorageClass mismatch.

The requested StorageClass may not exist, may not be the default, or may not support the requested access mode or size. Until a matching PersistentVolume is available or dynamically provisioned, the PVC will never bind, and the Pod will never start.

Always check the PVC status and Events before looking anywhere else.

Run:

```
kubectl describe pvc <pvc-name>
```

You may see Events like:

```
Events:
  Type      Reason             Age   From                                     Message
  ----      -
  Warning   ProvisioningFailed  1m    persistentvolume-controller            storageclass.storage.k8s.io "fast-ssd" not found
```

Or:

```
Warning ProvisioningFailed 2m persistentvolume-controller
failed to provision volume with StorageClass "standard": no available
volumes
```

Now, Verify the StorageClass exists or not. If yes, ensure the PVC requests a supported access mode and size, or set the correct default StorageClass if needed.

b. WaitForFirstConsumer Behavior

Some StorageClasses use **WaitForFirstConsumer**. This means the volume is not created or bound until the Pod is scheduled to a node.

This is expected behavior, but it looks broken. The PVC stays in **Pending** even though everything is configured correctly. Kubernetes is waiting so it can create the volume in the same zone as the node.

If the Pod itself cannot be scheduled, the PVC will also stay Pending. In this case, the real problem is scheduling, not storage.

Run:

```
kubectl describe pvc <pvc-name>
```

You may see:

```
Events:
  Type    Reason                Age   From                    Message
  ----    -
  Normal  WaitForFirstConsumer  2m    persistentvolume-controller waiting for first consumer to be created before binding
```

Now check the Pod:

```
kubectl describe pod <pod-name>
```

If you see scheduling errors like:

```
0/5 nodes are available: insufficient CPU
```

Then the PVC is waiting because the Pod cannot be scheduled.

To solve this, fix Pod scheduling issues(resources, taints, affinity). Then the Pod is scheduled, and the PVC binds automatically.

c. Attach and Detach Errors

Volumes are attached to nodes. If a node crashes or disappears, the volume may still be marked as attached to that node.

When Kubernetes tries to attach the same volume to a new node, you may see Multi-Attach errors. The Pod will stay stuck until the volume is detached from the old node.

```
Events:
  Type    Reason                Age   From                    Message
  ----    -
  Warning  FailedAttachVolume    2m    attachdetach-controller

Multi-Attach error for volume "pvc-xxx" Volume is already exclusively
attached to one node and can't be attached to another
```

This is common with cloud block storage. The fix is to wait for automatic detachment or manually clean up the attachment, depending on the storage provider.

At this point, it is important to understand what happens when a PersistentVolume is released after a PVC is deleted.

Every StorageClass defines a **reclaimPolicy**. This policy controls what happens to the PersistentVolume after the claim is deleted.

There are two common values:

- **Delete**: the underlying storage asset (for example, a cloud disk) is deleted automatically.
- **Retain**: the PersistentVolume object remains, and the underlying storage is kept. Manual cleanup is required.

*Many environments use the default **Delete** reclaimPolicy. This is convenient for temporary workloads, but it can be risky for critical stateful applications.*

If a production PVC is accidentally deleted and the StorageClass uses **Delete**, the underlying volume is permanently removed by the storage provider.

You can verify the reclaimPolicy by running:

```
kubectl get storageclass <storageclass-name> -o yaml
```

Look for:

```
reclaimPolicy: Delete
```

For critical data, especially databases, consider using **Retain** so that data is not automatically destroyed if a PVC is deleted by mistake.

This does not change how Pods mount volumes, but it changes the safety model of your storage lifecycle.

d. Permission Denied on Mounted Volumes

A very common storage issue is the application failing with “permission denied” errors after the volume is mounted.

This happens because the container runs as a non-root user, but the volume is owned by root. Kubernetes does not change file ownership by default.

The settings like fsGroup, runAsUser, and file permissions matter here. If they don't match, the Pod starts, but the application cannot read or write data and crashes immediately.



Part 6:

Security & Identity

In this part, we move from “can this run?” to “is this allowed to run?”.

This part focuses on identity (RBAC) and runtime security settings that commonly stop workloads in production.

Chapter 10: Access Denied (RBAC)

Every request in Kubernetes is evaluated against RBAC rules. Pods do not talk to the API server as users. They talk as ServiceAccounts. If a Pod cannot list, get, or update a resource, it means the ServiceAccount does not have permission.

The fastest way to debug this is with:

```
kubectl auth can-i
```

You can use it to ask Kubernetes a simple question: *Is this allowed or not?*

This works for users and for ServiceAccounts. It shows the real authorization decision.

A very common source of confusion is the difference between RoleBinding and ClusterRoleBinding.

A RoleBinding grants permissions only within one namespace, even if it points to a ClusterRole. This is what you want for application workloads.

A ClusterRoleBinding grants permissions across the entire cluster. This is powerful and dangerous if used carelessly.

Another common mistake is assuming the default ServiceAccount has permissions. By default, it has almost none. If a Pod suddenly starts failing API calls, always check which ServiceAccount it is using and what bindings exist for it.

First, identify which ServiceAccount the Pod is using:

```
kubectl describe pod <pod-name>
```

You may see:

```
Service Account: default
```

Then test permissions:

```
kubectl auth can-i list pods \
--as=system:serviceaccount:<namespace>:default
```

If the answer is:

```
no
```

That confirms an RBAC issue.

You can also see authorization failures in Events:

```
Events:
  Type    Reason      Age   From          Message
  ----    -
  Warning Forbidden   30s   kube-apiserver pods is forbidden: User "system:serviceaccount:example:default" cannot list resource "pods"
```

Always create a dedicated ServiceAccount for the app, bind only the permissions it needs, and prefer RoleBinding over ClusterRoleBinding whenever possible.

Chapter 11: Security Context Issues

This chapter focuses on the most common security-related failures that block workloads.

a. "Operation Not Permitted" and Missing Capabilities

Containers run with a very limited set of Linux capabilities by default. Many system-level operations are blocked on purpose. If an app tries to change the kernel parameters, network settings, or system time, it will fail and say "operation not permitted."

This means that the container needs an additional capability that was not granted. Some examples include network tools that require **NET_ADMIN** or time synchronization tools that need **SYS_TIME**.

If the app works outside of Kubernetes but not inside a container, check whether it relies on privileges that are not allowed by default.

Run:

```
kubectl describe pod <pod-name>
```

The Pod usually starts, but the container crashes. Logs may show:

```
Operation not permitted
```

There are usually no Kubernetes Events, because this is a runtime permission issue, not a scheduling or startup failure.

To solve this, identify which Linux capability the app needs, add only that specific capability using **securityContext.capabilities.add**, and avoid privileged containers unless absolutely necessary.

b. Read-Only Root Filesystems

The `readOnlyRootFilesystem` setting is a common security best practice. It prevents containers from writing to their root filesystem.

The bad thing is that many applications still try to write temporary files, logs, or caches to default paths. When they cannot write, they fail at runtime, even though the container starts successfully.

If an app breaks after enabling a read-only root filesystem, check where it writes files. Most of the time, the fix is to mount a writable volume for those paths.

Run:

```
kubectl describe pod <pod-name>
```

The Pod will usually be running, but restarting:

```
State:      Waiting
Reason:     CrashLoopBackOff
```

Check logs:

```
kubectl logs <pod-name>
```

You'll see file write errors:

```
ERROR: failed to create temporary file
open /tmp/app-cache.tmp: read-only file system

ERROR: cannot write log file
open /var/log/app.log: read-only file system
```

To avoid these, identify writable paths the app needs, mount an `emptydir` or persistent volume to those paths, and keep the root filesystem read-only.

c. Running as Non-Root and Port Binding

Running containers as non-root users is another security requirement. This improves isolation, but it comes with restrictions.

On Linux, binding to ports below 1024 requires root privileges. If an application tries to listen on port 80 or 443 while running as a non-root user, it will fail to start.

```
listen tcp :80: bind: permission denied
```

This shows up as a crash with no clear Kubernetes error. The solution is to use a higher port

inside the container and expose it through a Service, or adjust the image to support non-root execution.

d. Pod Security Standards (Admission-Level Security)

So far in this chapter, we discussed security issues that cause containers to crash after they start. Pod Security Standards are different. They can prevent a Pod from being created at all.

Kubernetes defines built-in Pod Security Standards that control what a Pod is allowed to do. These standards are enforced at the namespace level. If a Pod violates the enforced profile, Kubernetes rejects it before scheduling.

There are three predefined levels:

- **Privileged** – no restrictions
- **Baseline** – prevents known privilege escalation
- **Restricted** – enforces strict security controls for production workloads

Most production environments enforce restricted for application namespaces.

If your namespace has this label:

```
pod-security.kubernetes.io/enforce=restricted
```

And your Pod tries to run as root, use privileged mode, add forbidden capabilities, or mount hostPath volumes, Kubernetes will reject it.

You may see an error like:

```
Error from server (Forbidden):  
pods "app-backend" is forbidden:  
violates PodSecurity "restricted"
```

Or during deployment:

```
Warning FailedCreate replicaset-controller  
Error creating: pods "app-backend-" is forbidden: violates PodSecurity
```

This is not an RBAC issue. This is not a resource issue. This is an admission control decision.

To troubleshoot this:

First, check namespace labels:

```
kubectl get ns <namespace> --show-labels
```

Look for:

```
pod-security.kubernetes.io/enforce
```

Then inspect your Pod spec:

```
kubectl describe pod <pod-name>
```

Check for securityContext, runAsUser, Privileged, capabilities, hostNetwork and hostPath volumes.

Then compare what your Pod is requesting with what the enforced profile allows.

To fix this: modify the Pod to comply with the enforced standard, use a less strict profile (only if justified), or move the workload to a namespace with appropriate policy.



Part 7:

The Node Components

This section focuses on the node components you actually debug in real incidents.

Chapter 12: The Kubelet

The kubelet is the agent that runs on every node. It talks to the container runtime, manages Pods, mounts volumes, and reports node health. If the kubelet struggles, the entire node struggles.

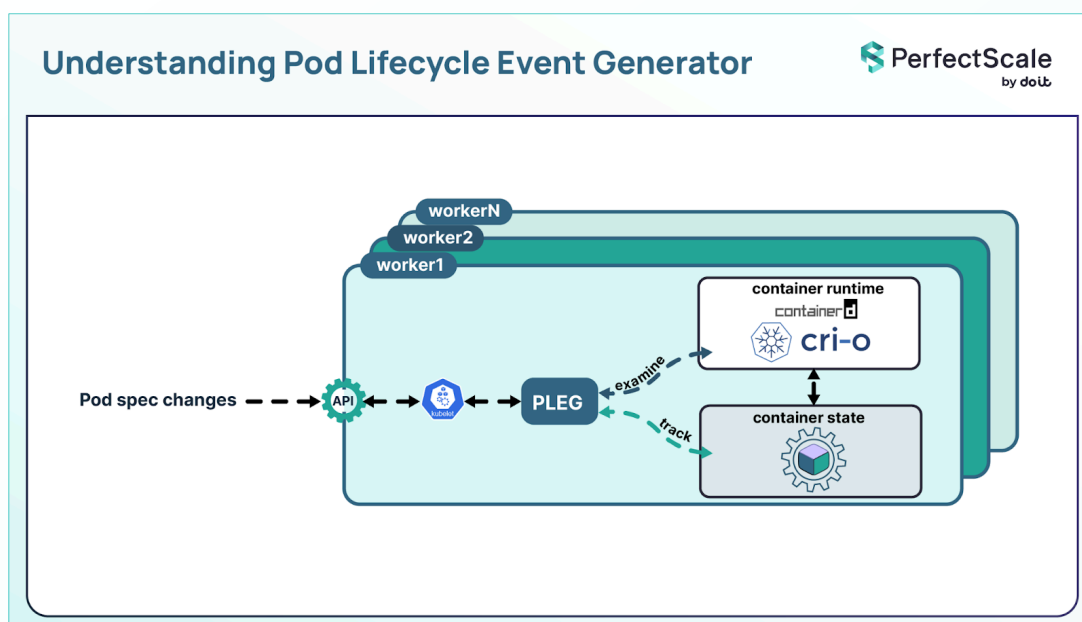
Most node-level problems show up here first.

a. PLEG Issues

PLEG (Pod Lifecycle Event Generator) is how the kubelet tracks what containers are actually doing. It asks the container runtime—containerd or CRI-O—about container state changes.

When the runtime is slow or overloaded, PLEG cannot keep up. The kubelet then starts logging warnings and may stop updating Pod status correctly. You may see Pods stuck in strange states, where restarts take longer than usual, or status updates are missing.

This happens under heavy load or when the container runtime is unhealthy.



b. Eviction Manager

When a node runs out of critical resources, the kubelet must evict Pods to protect itself. This is handled by the eviction manager.

Disk pressure is one of the most common triggers. Kubernetes tracks disk usage separately for **nodefs** (the node's root filesystem) and **imagefs** (where container images are stored).

If either fills up, Pods are evicted even if CPU and memory look fine.

PID pressure is another issue. Every process on the node consumes a PID. If the node runs out of available PIDs, Kubernetes cannot start new containers. Pods may fail or get evicted even though other resources are available.

These evictions are intentional. Kubernetes is trying to keep the node alive.

Also, when node behavior does not make sense, kubelet logs are the source of truth.

On most Linux systems using systemd, kubelet logs are available with:

```
journalctl -u kubelet
```

This is where you will see PLEG warnings, eviction decisions, runtime errors, and connectivity issues with the API server.

Chapter 13: Kube-Proxy & CNI

a. Kube-Proxy: IPTables vs IPVS

kube-proxy programs the node's networking rules so Services can forward traffic to Pods. It can run in iptables mode or IPVS mode.

iptables mode creates a large number of rules. It is simple and widely used, but on busy nodes, it can become slow to update. When rules are wrong or outdated, traffic may fail even though Services and Endpoints look correct.

IPVS mode uses kernel load balancing instead of long iptables chains. It scales better and is faster, but debugging is less familiar to many engineers. The important thing to know is that Service behavior depends on which mode kube-proxy is running in. When traffic behaves differently across nodes, check this first.

b. Debugging the CNI

The CNI plugin is responsible for assigning IPs and wiring networking for Pods. If CNI fails, Pods may stay in ContainerCreating or run without network access.

Every node stores its CNI configuration under:

```
/etc/cni/net.d/
```

If this directory is missing, misconfigured, or contains conflicting configs, Pod networking will break. This is one of the places to check when Pods cannot get IP addresses.

c. IP Address Exhaustion

CNI plugins allocate Pod IPs from a limited subnet. When that subnet runs out of addresses, new Pods cannot get networking. The Pod may be scheduled, but it will never become Ready.

This problem appears during scale-ups or rolling updates. The cluster looks healthy, but new Pods hang. The fix is not in the application; it's in the network design. Subnets must be sized for peak Pod count, not average/guesstimated usage.



Part 8:

Advanced and Rare Failures

This chapter covers problems that appear in production under real load.

These issues usually show up after the cluster has been running for a long time, under pressure, or during shutdowns and restarts.

Chapter 14: Zombies & Signals

a. Why PID 1 Matters?

PID 1 is the main process that runs inside a container. This process has special responsibilities. It must handle system signals and clean up child processes.

A lot of programs are not written to run as PID 1. When they are started directly, they may ignore signals or fail to clean up child processes. This causes serious problems during shutdown and restarts.

Using a proper init process, such as a lightweight init binary, makes sure signals are handled correctly and child processes are cleaned up.

b. Signals Propagation

When Kubernetes stops a Pod, it sends SIGTERM to allow the application to shut down gracefully. If the signal does not reach the real application process, the shutdown logic never runs.

Run:

```
kubectl delete pod <pod-name>
```

Watch the Pod status:

```
kubectl get pod <pod-name>
```

You may see the Pod stuck in:

```
Terminating
```

Check Events:

```
Events:
  Type    Reason      Age   From      Message
  ----    -
  Warning  Killing     30s   kubelet   Container failed to stop in time
```

Eventually, Kubernetes force-kills the container.

This happens when a shell script starts the app and does not forward signals. Kubernetes thinks it asked the app to stop, but the app never hears the request.

The result is slow termination, forced kills, and sometimes data loss.

To avoid this, prevent wrapping applications in shell scripts that swallow signals, use **exec** form in container entrypoints, or use an init process that forwards signals correctly.

c. Zombie Processes and Node Instability

If child processes exit but are never cleaned up, they become zombies. Zombies do not use CPU or memory, but they consume PIDs.

Over time, a node can run out of available PIDs. When that happens, new containers cannot start. Pods fail for no obvious reason, and the node becomes unstable.

This problem is hard to find unless you know what to look for, and it points back to missing or broken init behavior.

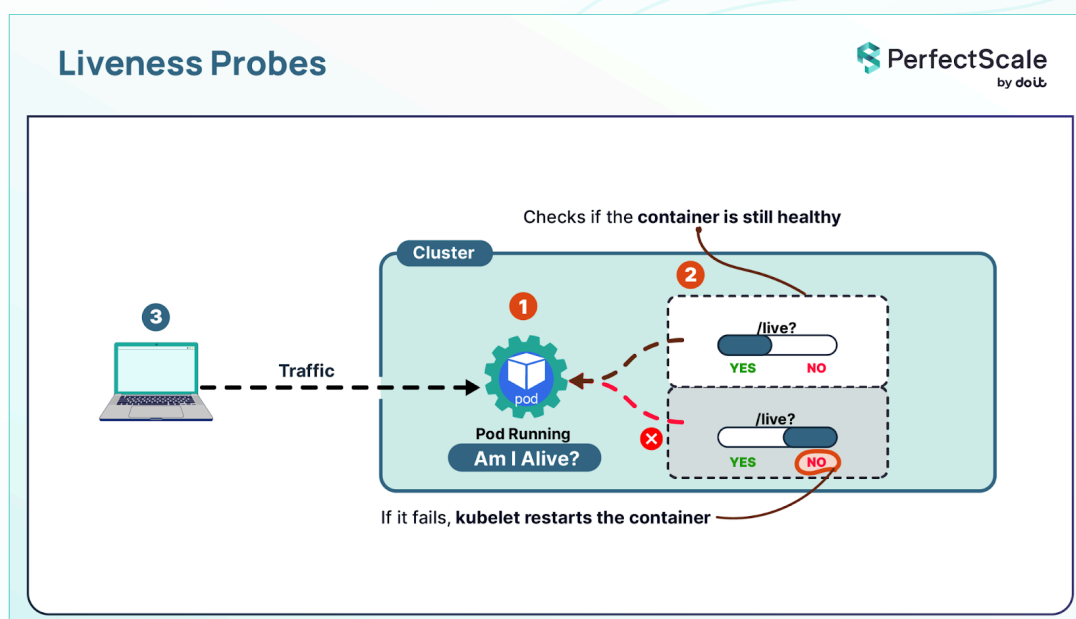
d. Liveness Probes

Liveness probes are meant to detect broken applications. If misused, they can become the cause of outages.

Exec-based probes are a common issue. Every probe execution starts a new process. If those processes are not cleaned up properly, they can contribute to zombie buildup.

Aggressive probe settings can also slow applications. Under high load, a slow response is treated as failure. Kubernetes restarts the Pod, making the situation worse.

One recommended way to avoid these issues is to use a small init process such as **tini**. Tini acts as a proper PID 1 inside the container. It forwards signals correctly and cleans up zombie processes.





Part 9:

Real-World Case Studies

So far in this book, you learned how to troubleshoot Kubernetes problems by reading Events, understanding Pod states, checking resources, networking, and node behavior. All of this might feel theoretical until you see what actually happens when these basics are missed in production.

Publicly reported incidents show the same patterns. In each case, Kubernetes did exactly what it was told to do, but small misconfigurations, overlooked defaults, or unsafe changes caused widespread outages.

Case Study 1:

Reddit's Pi-Day Outage

On March 14, 2023 – [Pi Day – Reddit](#) went through a major outage that lasted more than five hours. Large parts of the site became unavailable. What made this incident painful was that it did not start with a bug or traffic spike. It started with a routine Kubernetes upgrade on a production cluster.

During the upgrade, Kubernetes removed the old **master** node label and replaced it with **control-plane**. This small change had a big impact. The networking components (Calico) relied on the old label for scheduling and routing. As a result, Pods lost network connectivity, DNS resolution started failing, and services that depended on each other could no longer communicate.

So, to prevent these types of outages, you should always know how to check service endpoints, inspect DNS behavior, and look at CNI-related Events early, so you can narrow down the problem much faster than we discussed in our ebook earlier.

Case Study 2:

Preply's Partial DNS Outage

In early 2020, [Preply](#) experienced a partial outage where several services became unreliable. Requests failed intermittently, but there were no obvious application crashes. Pods were running, CPU and memory looked fine, and nothing appeared “down” at first glance.

The real issue was DNS. CoreDNS, which handles name resolution inside Kubernetes, was being autoscaled incorrectly. The autoscaler reduced the number of CoreDNS Pods too aggressively. Under load, there simply weren't enough DNS servers to handle requests. Some lookups failed, others timed out, and services could not reliably find each other.

This is a perfect example of why DNS problems are so dangerous.

Case Study 3:

Free Now Node Bootstrap Failure

In September 2019, the ride-sharing company [Free Now](#) faced an incident where workloads stopped scaling correctly. New Pods stayed stuck in Pending. There were no obvious errors in application logs.

The root cause was not the pods at all; it was the nodes. A change in the node bootstrap image introduced a dependency on a hard-coded CentOS mirror. When that mirror became unreachable, new worker nodes failed to finish their setup. They joined the cluster partially but were never truly ready to run workloads.

As a result, the scheduler kept trying to place Pods on nodes that could not actually accept them. Pods stayed Pending, and the cluster slowly backed up.

All three incidents share a simple truth: Kubernetes was behaving correctly, but the systems around it were misunderstood, misconfigured, or changed without fully understanding the impact.

In every case, Kubernetes produced signals: failing DNS, missing endpoints, Pending Pods, broken networking, but those signals only help if you know how to read them.

That is the purpose of this ebook.

By learning how to troubleshoot Pods, networking, DNS, autoscaling, and nodes step by step, you reduce the time between when *something feels wrong* and *you know exactly where to look*.



Part 10:

Preventing These Problems at Scale

By now, you can troubleshoot most Kubernetes incidents faster than the average team because you know where the signal lives: Events, logs, resource behavior, and node components.

But production has two realities:

1. Kubernetes changes constantly

Deployments roll, nodes churn, autoscalers react, and one “small” YAML change can introduce instability.

2. Manual troubleshooting is reactive

By the time you’re debugging a `CrashLoopBackOff` or a node under pressure, you’re already in the blast radius.

Preventing incidents isn’t about never having failures. It’s about catching the patterns early and reducing how often you enter incident mode.

This is where platforms like **PerfectScale by DoiT** fit into a modern Kubernetes operations workflow: they turn raw usage signals into prioritized risks, recommendations, and (optionally) safe automation.

PerfectScale adds a continuous “analysis layer” over your clusters:

- **Finds resource misconfiguration** that leads to Pending Pods, throttling, OOMKills, and scaling instability
- **Surfaces high-risk workloads** before they become outages
- **Highlights infrastructure inefficiency** that increases noisy-neighbor problems and eviction Events
- **Tracks trends over time** so you can spot growing risk instead of reacting to one-off spikes
- **Optionally automates** safe adjustments so right-sizing doesn't become endless toil.

You can still troubleshoot with `kubectl` (and you should). The point is to reduce how often you need to.

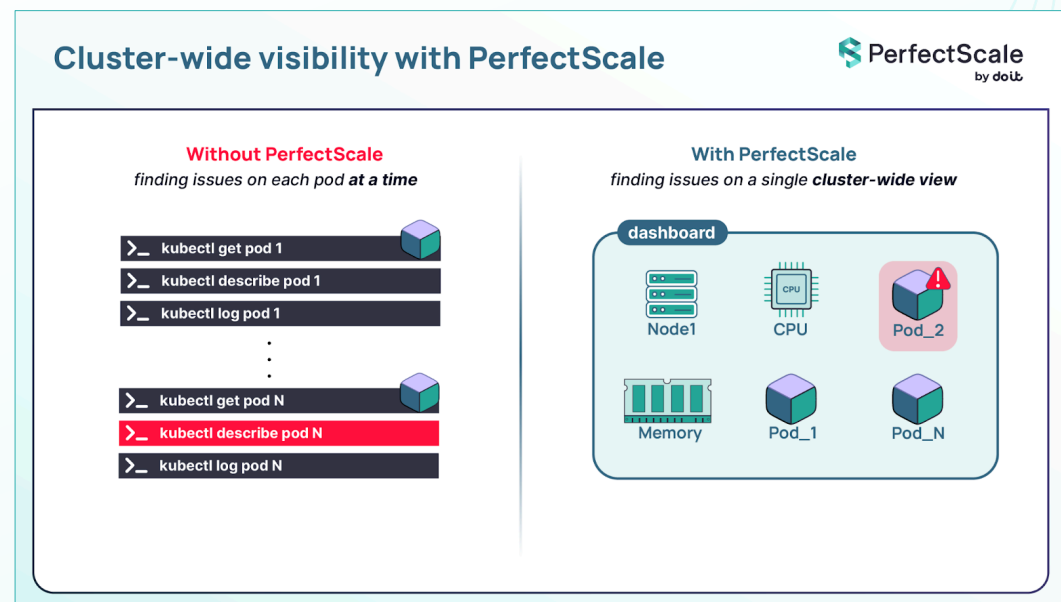
How PerfectScale Helps You Avoid Troubleshooting Pain?

Here's what PerfectScale does that directly connects to the kinds of problems you learned about earlier in this book:

1. Cluster-wide visibility (so you don't debug one Pod at a time)

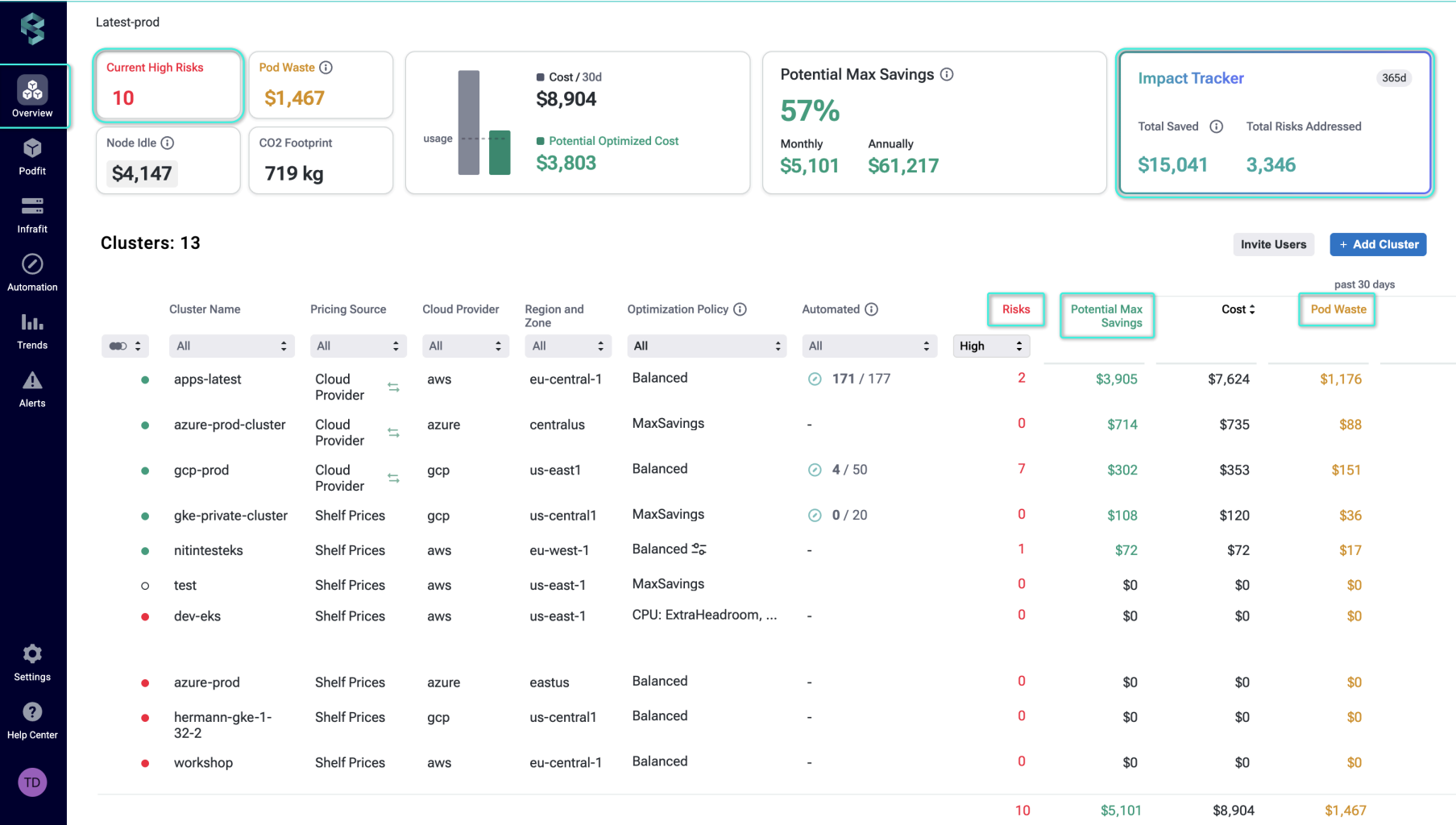
PerfectScale gives you a single, cluster-wide view of your Kubernetes environment through the Overview dashboard.

You can immediately see where risk, waste, and instability exist across clusters.



High-risk workloads lead to CrashLoopBackOff or OOMKills, while Pod waste usually comes from incorrect requests and limits. Instead of finding these issues one Pod at a time, you see them across the entire platform.

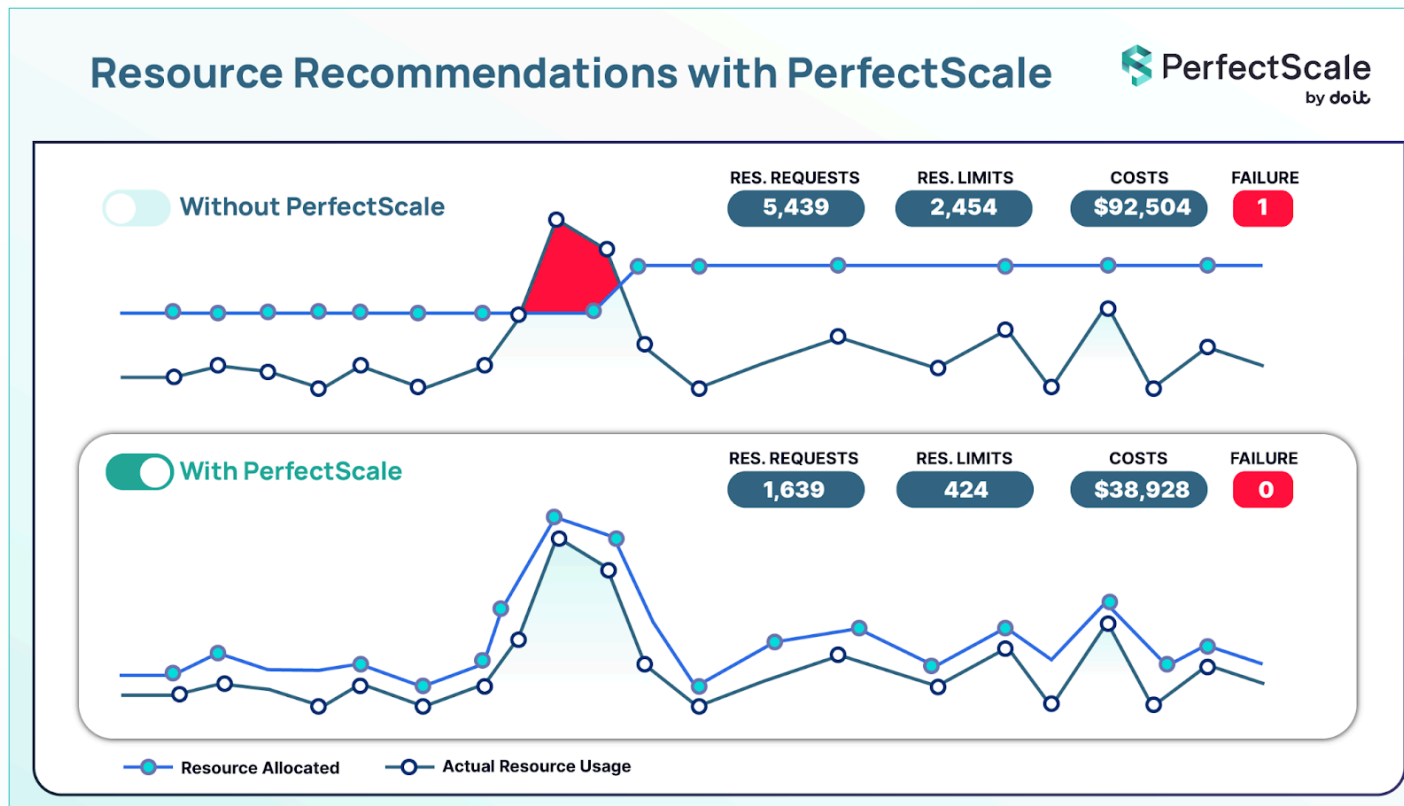
The operational advantage: you start with “**what’s most risky right now?**”, not “which Pod should I click next?”



2. Resource recommendations (to prevent the “requests vs limits” disasters)

A large portion of production incidents comes down to resource settings that don't match reality:

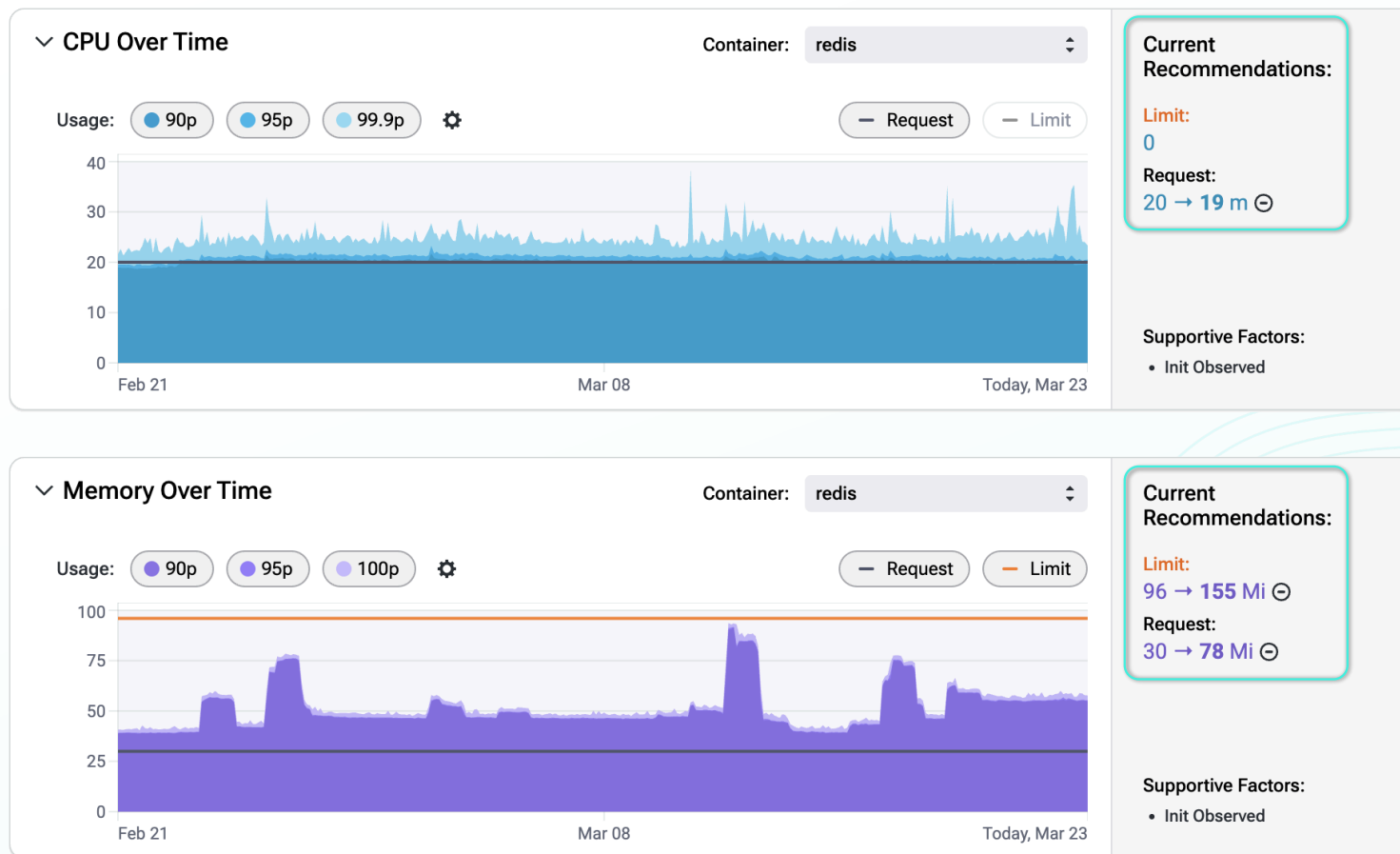
- requests too high → Pods stuck Pending, poor bin packing
- limits too low → OOMKills, throttling-induced latency
- missing requests → BestEffort evictions and noisy-neighbor chaos



PerfectScale's workload analysis (often referred to as "right-sizing") helps you identify where requests/limits are:

- oversized (waste + scheduling pain)
- undersized (risk of throttling/OOM)
- unstable (patterns that often cause HPA flapping)
- In-Place Pod Resize (safer, faster optimization without disruption)

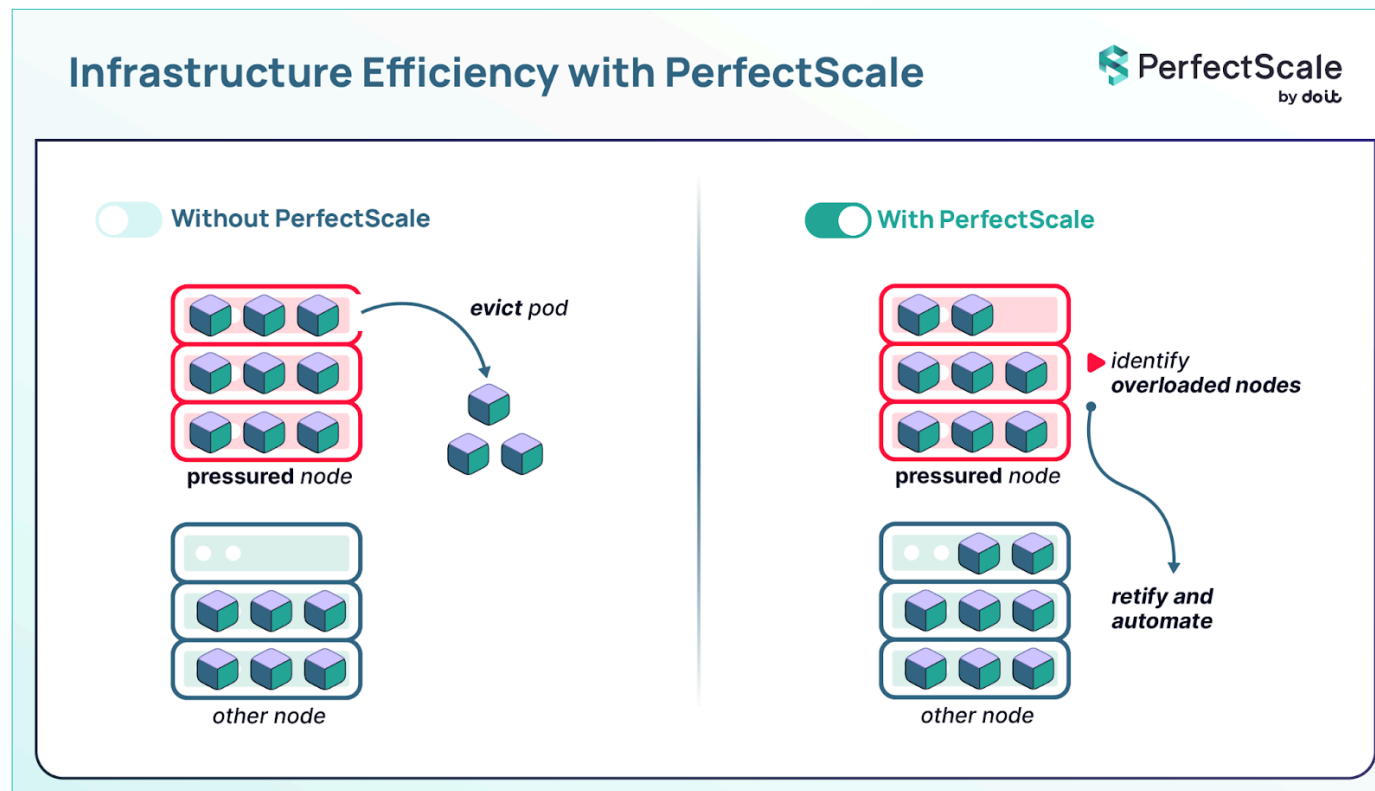
This directly reduces failures covered in Chapter 4 (Requests vs Limits) and Chapter 6 (Autoscaling problems).



3. Infrastructure Efficiency (to reduce node-level instability)

Many “mysterious” issues aren’t mysterious; they’re the node quietly running out of something:

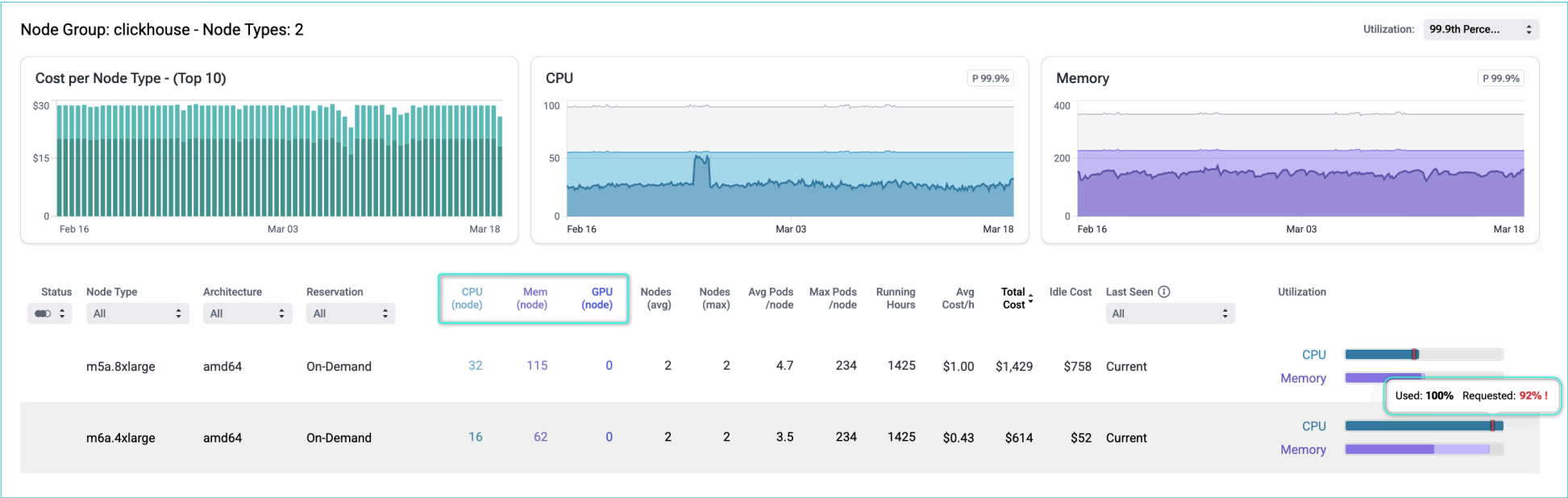
- memory pressure → evictions
- disk pressure → evictions
- IP exhaustion → Pods stuck ContainerCreating
- uneven bin packing → noisy neighbors and random latency



PerfectScale’s infrastructure efficiency views help you understand:

- where capacity is stranded
- where certain nodes are overloaded
- where autoscaling policies create churn instead of stability

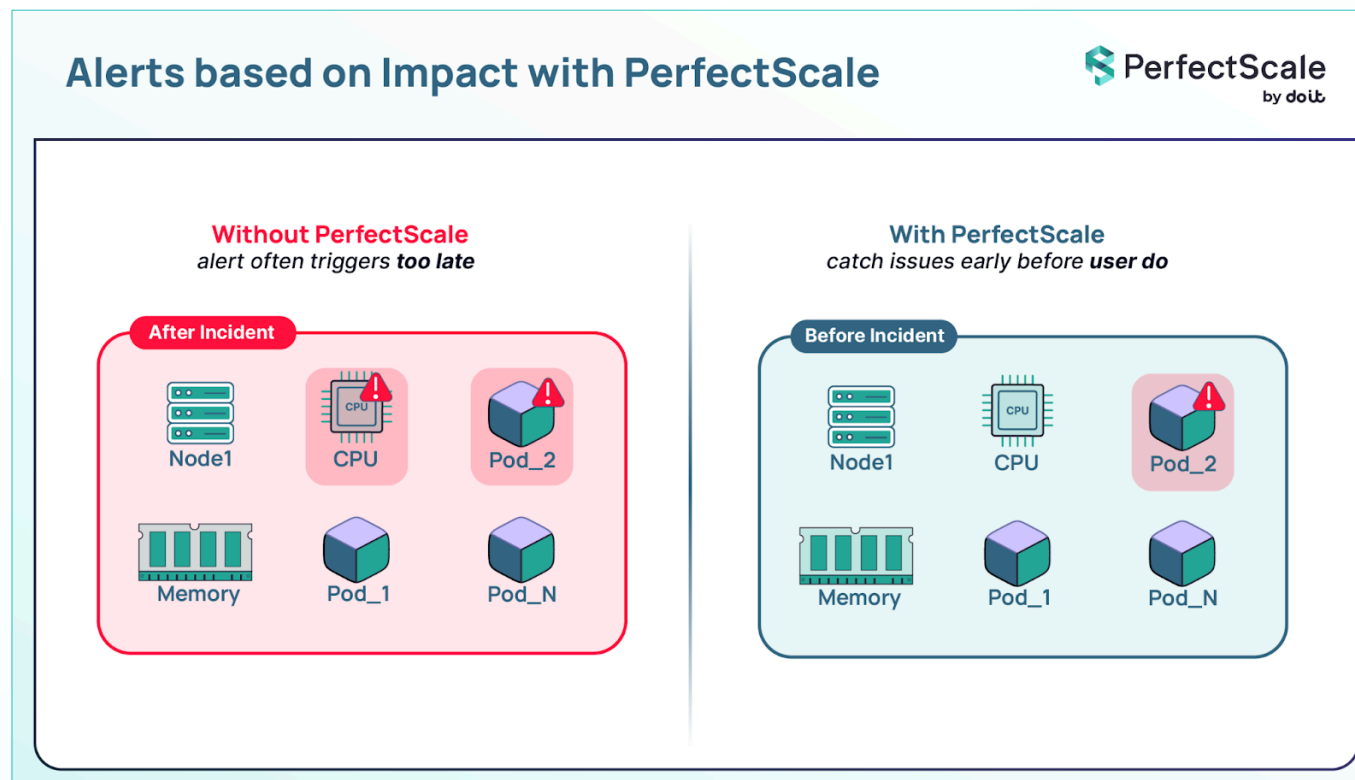
The operational advantage: fewer “random” node incidents and fewer fire drills caused by cluster drift.



4. Alerts based on impact (so you catch trouble before users do)

Traditional alerting often triggers too late:

- "Pod is down." → you're already in incident mode
- "Node not ready" → you're already losing capacity



PerfectScale's more useful alerts are early warnings:

- memory usage approaching limit (risk of OOMKill)
- sustained CPU throttling (performance degradation)
- workloads oscillating (autoscaling instability)
- configurations that repeatedly correlate with incidents (the “known-bad patterns”)

If you integrate with Slack / Teams, the key is not “more alerts.” It's fewer, higher-signal alerts that tell you what to look at first.

Latest-prod / Alerts / Resilience

▲ Active Alerts
74

▲ Resilience
22

💰 Cost
52

🗑️ Waste
0

Resilience · 38 Financial · 119

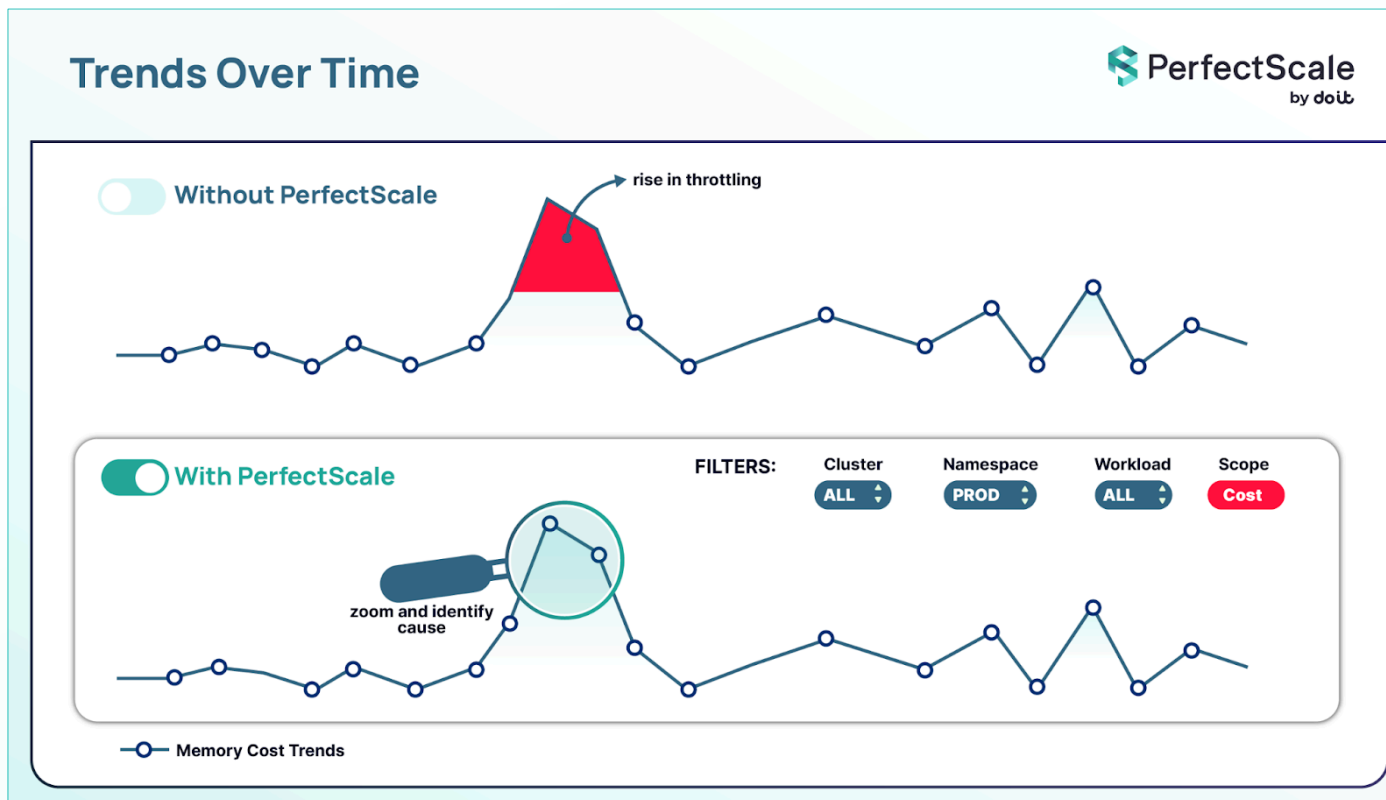
Status	Alert Details	Cluster	Namespace	Workload	Type	Container	First Seen	Last Seen
●●●	Mem Limit Not Set	apps-latest	monitoring	victoria-logs-victoria-logs-single-server	StatefulSet	vlogs	Jan 26, 2026, 1:47 AM	Mar 25, 2026, 11:48 AM
●●●	Mem Request Not Set	apps-latest	monitoring	victoria-logs-victoria-logs-single-server	StatefulSet	vlogs	Jan 26, 2026, 1:47 AM	Mar 25, 2026, 11:48 AM
●●●	Under-Provisioned Cpu Request	apps-latest	apps	psc-aggregator-reaggregation-scaledjob	ScaledJob	psc-aggregator-scaled-job	Aug 10, 2023, 12:30 AM	Mar 25, 2026, 11:48 AM
●●●	Mem Limit Not Set	nitintesteks	eks-sample-app	eks-sample-linux-deployment	Deployment	nginx	Jul 29, 2025, 10:44 PM	Mar 25, 2026, 11:48 AM
●●●	Cpu Request Not Set	gcp-prod	vk-test-gke	update-ecr-secret	CronJob	updater	Jan 31, 2026, 12:44 AM	Mar 25, 2026, 11:48 AM
●●●	Restarts Observed	gcp-prod	onprem	onprem-exporter	Deployment	exporter	May 16, 2025, 4:33 PM	Mar 25, 2026, 11:48 AM
●●●	Restarts Observed	gcp-prod	loki	loki	StatefulSet	loki	Mar 18, 2024, 10:30 PM	Mar 25, 2026, 11:48 AM
●●●	Restarts Observed	gcp-prod	automation	automation-memory-under-provisioned	Deployment	automation-memory-under-provisioned	Mar 18, 2024, 11:30 PM	Mar 25, 2026, 11:48 AM
●●●	OOM	gcp-prod	automation	automation-memory-under-provisioned	Deployment	automation-memory-under-provisioned	Mar 18, 2024, 10:30 PM	Mar 25, 2026, 11:48 AM
●●●	Under-Provisioned Mem Request	gcp-prod	automation	auto-memory-under-prov-node-oom	Deployment	auto-memory-under-prov-node-oom	Mar 18, 2024, 10:30 PM	Mar 25, 2026, 11:48 AM
●●●	Mem Limit Not Set	gcp-prod	automation	auto-memory-under-prov-node-oom	Deployment	auto-memory-under-prov-node-oom	Mar 18, 2024, 10:30 PM	Mar 25, 2026, 11:48 AM
●●●	Mem Request Not Set	gcp-prod	monitoring	prometheus-prometheus-kube-prometheus-prometheus	StatefulSet	prometheus	Feb 17, 2025, 4:32 PM	Mar 25, 2026, 11:48 AM
●●●	Mem Limit Not Set	gcp-prod	monitoring	prometheus-prometheus-kube-prometheus-prometheus	StatefulSet	prometheus	Oct 23, 2024, 3:31 PM	Mar 25, 2026, 11:48 AM
●●●	Restarts Observed	gcp-prod	primary	primary-under	Deployment	primary-under	Nov 25, 2024, 4:30 PM	Mar 25, 2026, 11:48 AM
●●●	OOM	gcp-prod	primary	primary-under	Deployment	primary-under	Oct 23, 2024, 7:31 AM	Mar 25, 2026, 11:48 AM

5. Trends over time (to avoid slow-motion outages)

Some incidents don't start as incidents.

They start as:

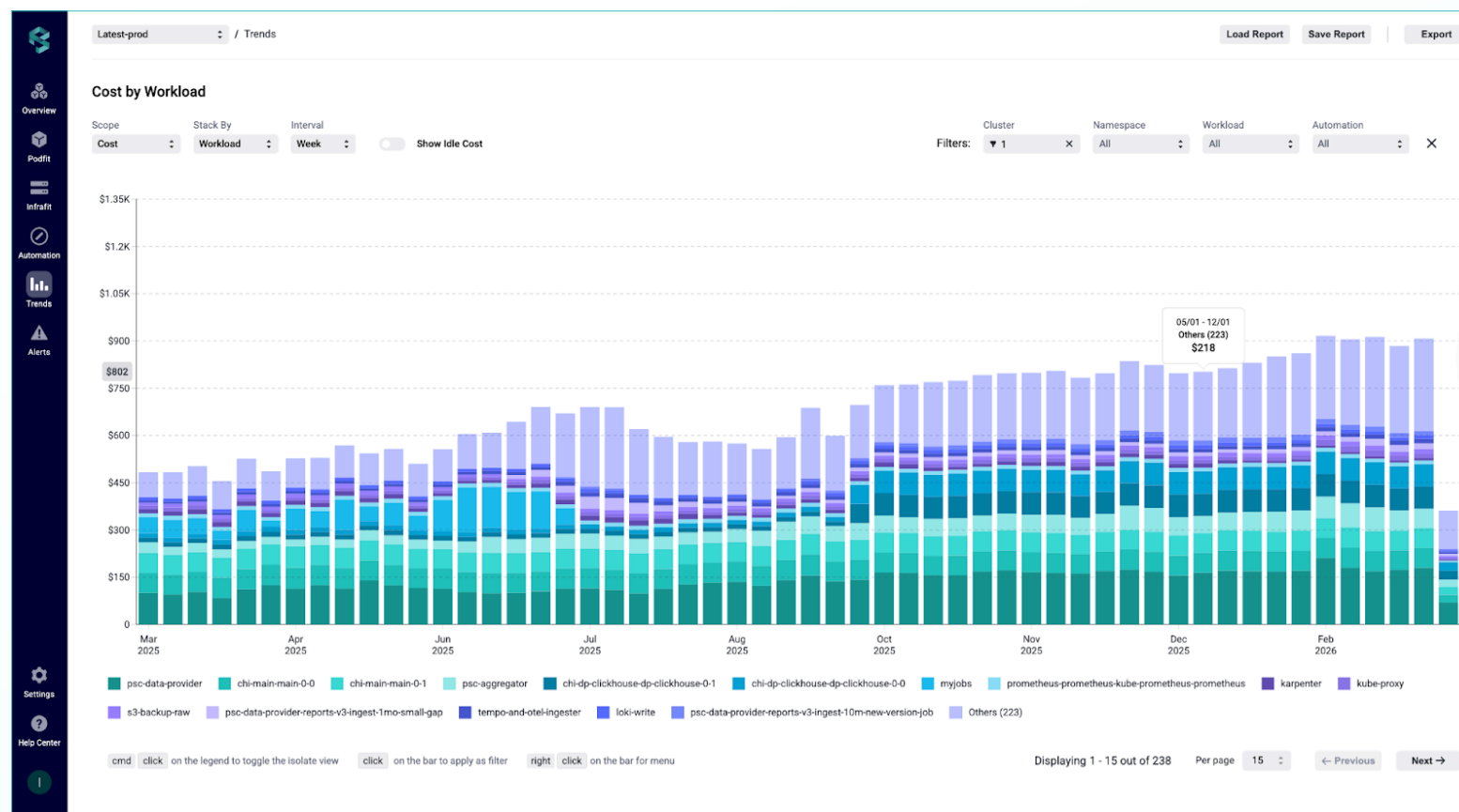
- a slow increase in memory usage week-over-week
- a gradual rise in throttling
- "waste" is rising because requests were set once and never revisited.



Trends view helps you answer:

- Is this a one-off spike or a real shift?
- Which namespace/workload is driving it?
- Did a deployment correlate with the change?

This is how you move from reactive troubleshooting to proactive operations.

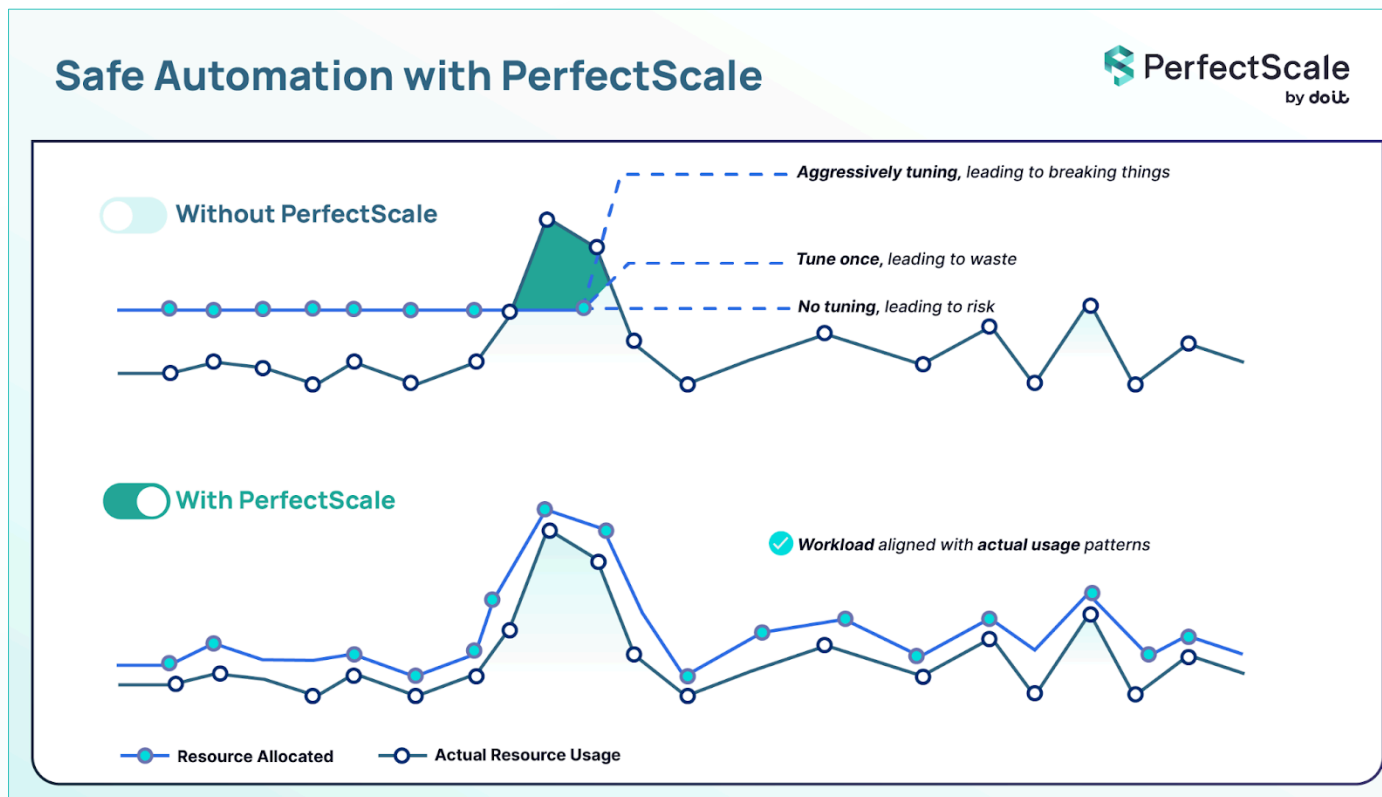


6. Safe automation (so right-sizing doesn't become a monthly project)

Manual tuning doesn't scale. Teams either:

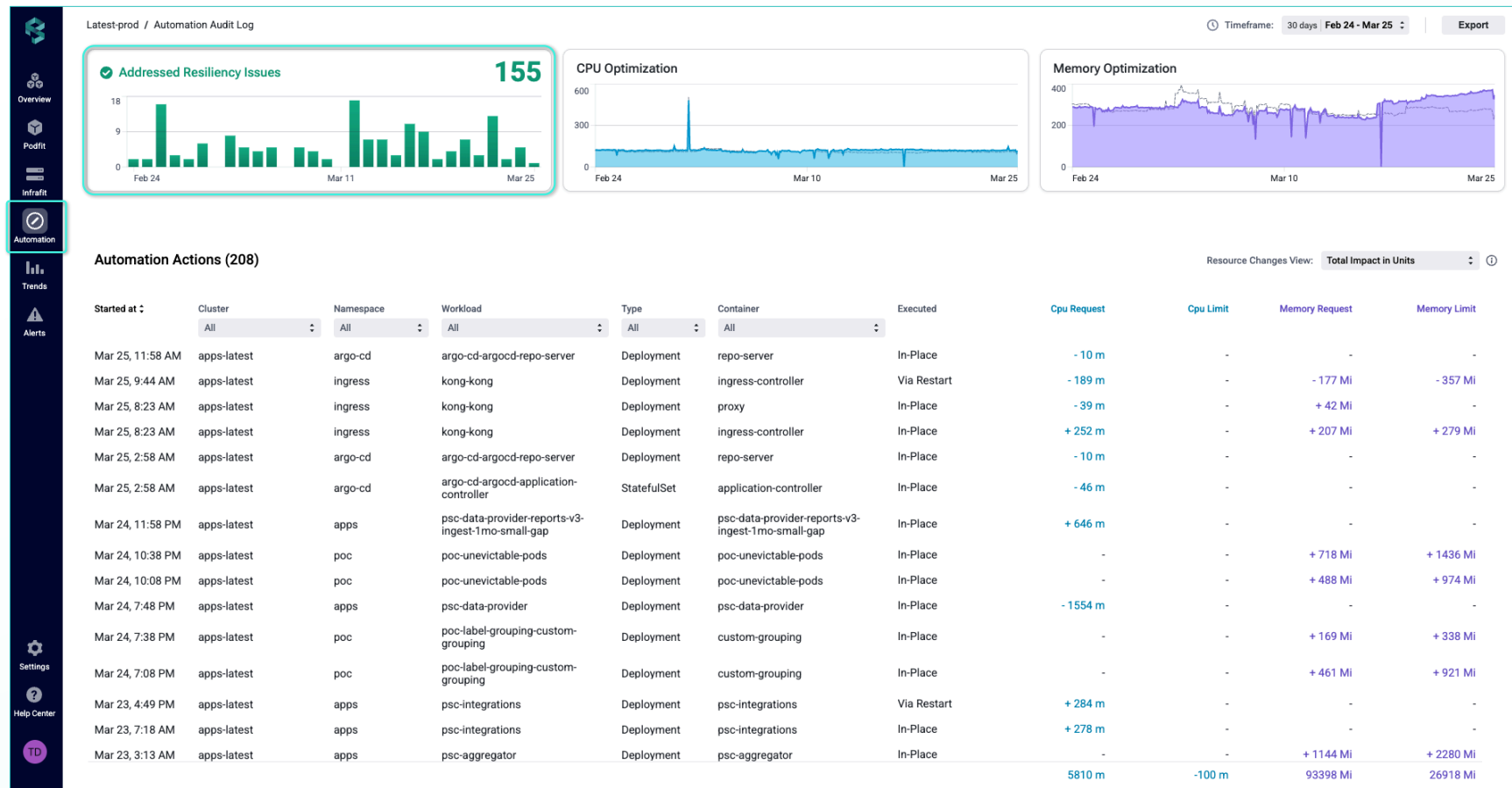
- don't do it (risk + waste grows)
- do it once (then drift returns)
- do it aggressively (then accidentally break things)

Automation can help keep workloads aligned with actual usage patterns *without constant manual edits*.



Operationally, the safest approach is:

- start with recommendations only
- apply changes gradually
- validate health signals (error rate, latency, restarts)
- automate only after you trust the behavior



So, now that's a question: how do you onboard your first Cluster?

That's so easy!!!

PerfectScale is very easy to set up in your Kubernetes environment:

1. Deploy the PerfectScale agent via Helm into your cluster.
2. Within minutes, PS starts collecting metrics and analyzing workload behavior.
3. You start seeing recommended actions in the dashboard, including optimization insights and risk indicators.
4. Start eliminating your first “high-signal” wins like top OOMKill workloads, biggest sustained throttling offenders, or highest waste workloads and cut your K8s cloud spending by up to 50%.

Reduce your cloud bill and improve application performance today

Install in minutes and instantly receive actionable intelligence.

Start for free

Request a demo